



자율 에이전트를 활용한 AI 네이티브 자바 개발

AWS 테크에반젤리스트
최용호

Agenda

1. AI 에이전트로 빠르게 자바 개발
2. AI 에이전트로 팀 전체의 코딩 품질 끌어올리기
3. 자율 에이전트로 더 안정적인 개발

AI 에이전트로 빠르게 자바 개발



AI가 소프트웨어를 변화시키고 있습니다

2024

어시스턴트

개발자가 더 빠르게 코드를
작성할 수 있도록 지원

2025

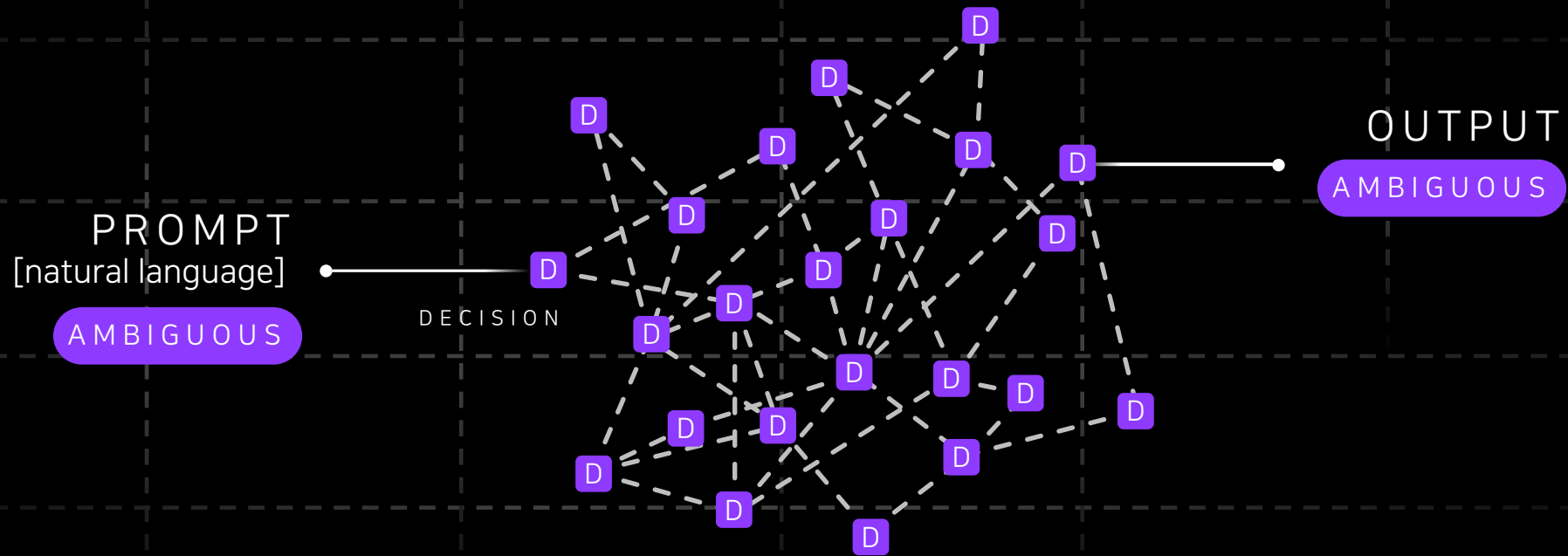
에이전트

사람이 질문하고 AI가
답하는 방식으로 상당량의
코드를 AI가 개발

2026

자율성

End-to-end AI가
자율적으로 개발하고, 필요
시 사람이 개입 (Human in
the loop)



Prompt
[natural language]

AMBIGUOUS



SPEC

PRECISE

DECISION

OUTPUT
PRECISE



개발자처럼 코드 작성 전에 문서 먼저 작성하도록 하려면?



Claude Code Plan mode

```
> /plan
```

/plan	Enable plan mode or view the current session plan
/batch	Research and plan a large-scale change, then execute it in parallel across 5-30 isolated worktree agents that each open a PR.
/usage	Show session cost, plan usage, and activity stats
/claude-api	Reference for the Claude API / Anthropic SDK – model ids, pricing, params, streaming, tool use, MCP, agents, caching, token counting, model migration. TRIGGER – read BEFORE ...



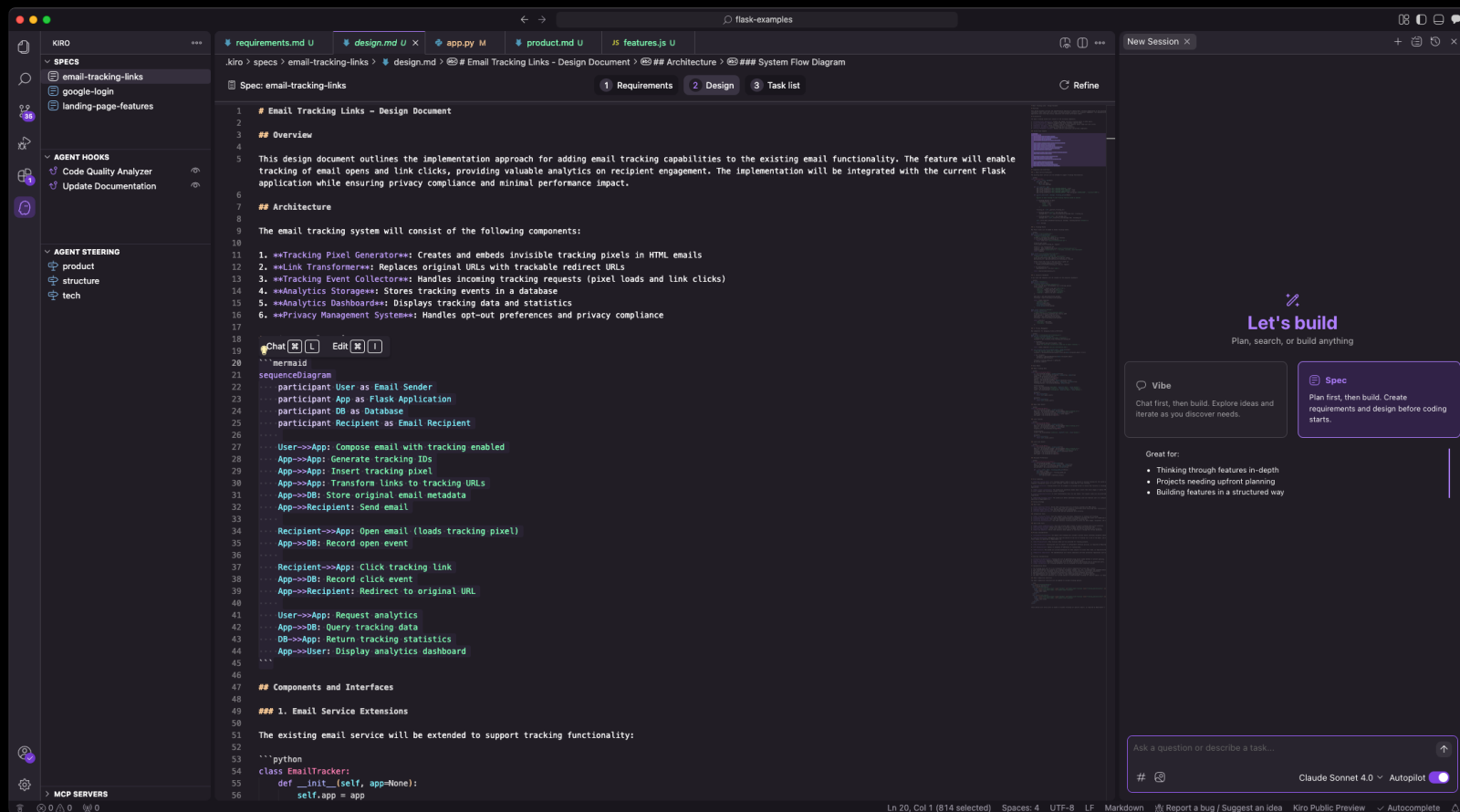
Codex Plan mode

- Model changed to gpt-5.5 medium for Plan mode.

```
> 온라인 쇼핑물을 구현해줘
```

- 요청하신 작업은 현재 Plan Mode라서 바로 구현하지 않고, 먼저 저장소 구조와 기술 스택을 확인한 뒤 구현 계획을 확정하겠습니다. 지금은 어떤 프로젝트 위에 쇼핑물을 얹을 수 있는지부터 살펴보겠습니다.

KIRO 프로토타입에서 프로덕션까지 가능한 AI IDE



Kiro는 Spec-driven 개발을 통해 AI 코딩에 구조를 부여하여 최상의 작업을 수행할 수 있도록 도와줍니다.

Kiro IDE 기능 기반 사양 (Spec) 흐름

"바이브 코딩"의 모호함을 넘어, 정밀한 사양(Specification)을 중심으로 AI와 소통



STEP 01

요구사항 Requirements



사용자 관점에서의 목적과 역할 명확화
해결해야 할 핵심 문제와 기대 결과 정의

STEP 02

설계 Design



요구사항 충족을 위한 기술적 접근 방식
시스템 구조와 컴포넌트 상호작용 설계

STEP 03

작업 Task



설계 구현을 위한 구체적 코드 변경 사항
실행 단계 정의를 통해 AI가 코드 생성

```
> .vscode
PRD.md

## # 피부 진단 및 화장품 추천 웹 서비스 개발을 위한 상세 요구사항 개선안 > ## ## 2. 피부 유형 분류 및 진단 질문 설계 :
1 # 피부 진단 및 화장품을 추천 웹 서비스 개발을 위한 상세 요구사항 개선안
2
3 ## 1. 프로젝트 개요
4
5 ### 1.1 서비스 목적
6 - **사용자 피부 진단** : 10개의 체계적인 질문을 통해 사용자의 피부 유형을 정확히
  진단
7 - **개인 맞춤형 제품 추천** : Amazon Bedrock을 활용한 AI 기반 피부 분석
  리포트 및 화장품 추천
8 - **사용자 친화적 인터페이스** : 간단한 버튼 클릭 방식의 직관적인 사용자 경험 제공
  [1] [2] [3]
9
10 ### 1.2 기술 스택
11 - **프론트엔드** : 반응형 웹 인터페이스 (HTML5, CSS3, JavaScript)
12 - **백엔드** : 질문 로직 처리 및 API 연동
13 - **AI 서비스** : Amazon Bedrock을 통한 피부 진단 분석 및 리포트 생성 [2] [4]
  [5]
14
15 ## 2. 피부 유형 분류 및 진단 질문 설계
16
17 ### 2.1 피부 유형 정의
18 현대적이고 과학적인 접근을 위해 **바우만 피부 타입 분류법**을 기반으로 한 4가지
  기본 피부 유형으로 분류: [6] [1]
19
20 1. **건성 피부 (Dry Skin)**
21 - 특징: 피지 분비량이 적고, 수분 부족으로 인한 당김 현상
22 - 주요 고민: 각질, 주름, 탄력 저하
23
24 2. **지성 피부 (Oily Skin)**
25 - 특징: 피지 분비량이 많고, 번들거림과 넓은 모공
26 - 주요 고민: 여드름, 블랙헤드, 모공 확장
27
28 3. **복합성 피부 (Combination Skin)**
29 - 특징: T존(이마, 코, 턱)은 지성, U존(볼, 눈가)은 건성
30 - 주요 고민: 부위별 차별적 관리 필요
31
32 4. **민감성 피부 (Sensitive Skin)**
33 - 특징: 외부 자극에 쉽게 반응, 홍조와 가려움 발생
34 - 주요 고민: 트러블, 알레르기 반응, 염종 [7] [8] [6]
35
36 ### 2.2 피부 진단 질문 설계 (10개 질문, 각 2개 선택지)
37
38 **질문 1: 세안 후 피부 느낌**
39 - A) 땅기고 건조한 느낌이 든다 (건성 +1)
40 - B) 촉촉하거나 번들거린다 (지성 +1)
41
42 **질문 2: 평소 피부 상태**
43 - A) 각질이 자주 일어난다 (건성 +1)
44 - B) 기름이 많이 난다 (지성 +1)
45
46 **질문 3: T존(이마, 코) 상태**
```

Let's build

Plan, search, or build anything

Vibe

Chat first, then build. Explore ideas and iterate as you discover needs.

Spec

Plan first, then build. Create requirements and design before coding starts.

Great for:

- Thinking through features in-depth
- Projects needing upfront planning
- Building features in a structured way

Ask a question or describe a task...

#

Claude Sonnet 3.7 v Autopilot

Design 문서 생성

PRD.md

requirements.md x

🔍 📄 ⋮

New Session x

+ 📄 🔄 ✕

SPECS

📄 skin-diagnosis-service

AGENT HOOKS

Automate repetitive tasks with smart triggers

AGENT STEERING

Guide agent behavior and responses

Generate Steering Docs

MCP SERVERS

Connect external tools and data sources

Enable MCP

.kiro > specs > skin-diagnosis-service > requirements.md > # 요구사항 문서 > ## 요구사항

📄 Spec: skin-diagnosis-service 1 Requirements 2 Design 3 Task list 🔄 Refine

```
1 # 요구사항 문서
7 ## 요구사항
85 ### 요구사항 8
88
89 ##### 승인 기준
90
91 1. WHEN 웹 페이지가 로드되면 THEN 시스템은 반응형 디자인으로 다양한 화면 크기에
    적응해야 합니다
92 2. WHEN 모바일에서 접근하면 THEN 시스템은 터치 친화적인 인터페이스를 제공해야
    합니다
93 3. WHEN 페이지가 로딩되면 THEN 시스템은 3초 이내에 콘텐츠를 표시해야 합니다
94 4. WHEN API 요청이 발생하면 THEN 시스템은 5초 이내에 응답을 반환해야 합니다
```

PRD.md 파일에 작성된 요구사항에 따라 코드를 구현해주세요. 이 때 개발 언어로는 Python을 사용하고 API 구현을 위해 FastAPI 프레임워크를 사용합니다. 사용자 데이터는 아직은 저장할 필요가 없어서 데이터는 메모리에 저장해주세요.

PRD.md 파일에 작성된 요구사항에 따라 코드를 구현해주세요. 이 때 개발 언어로는 Python을 사용하고 API 구현을 위해 FastAPI 프레임워크를 사용합니다. 사용자 데이터는 아직은 저장할 필요가 없어서 데이터는 메모리에 저장해주세요.

👤 Kiro

PRD.md 파일을 먼저 읽어서 요구사항을 확인하고, 이를 바탕으로 spec을 생성하겠습니다.

📄 Read file(s) requirements.md

PRD.md 파일을 확인했습니다. 피부 진단 및 화장품 추천 웹 서비스에 대한 상세한 요구사항이 작성되어 있네요. 이를 바탕으로 spec을 생성하겠습니다.

먼저 feature name을 "skin-diagnosis-service"로 정하고 requirements.md 파일을 생성하겠습니다.

📄 Created requirements.md

요구사항 문서를 작성했습니다. PRD.md의 내용을 바탕으로 8개의 주요 요구사항을 EARS 형식으로 정리했습니다. 각 요구사항은 사용자 스토리와 구체적인 승인 기준을 포함하고 있습니다.

요구사항이 괜찮아 보이나요? 그렇다면 설계로 넘어갈 수 있습니다.

Move to design phase

Ask a question or describe a task...

📄

Claude Sonnet 3.7 v Autopilot 🔴

Task list 문서 생성

design.md ↔ design.md /.koro/specs/skin-diagnosis-service

.koro > specs > skin-diagnosis-service > design.md > # 설계 문서 > ## 개요

```
1 # 설계 문서
33 ## 컴포넌트 및 인터페이스
76 ### 프론트엔드 컴포넌트 (React)
99 async submitAnswer(sessionId, questionnaire, answer,
100 async getResult(sessionId)

83+ ##### 2. 페이지 컴포넌트
84+ ```typescript
85+ // HomePage.tsx - 메인 페이지
86+ interface HomePageProps {}
87+ const HomePage: React.FC<HomePageProps>
88+
89+ // DiagnosisPage.tsx - 진단 페이지
90+ interface DiagnosisPageProps {}
91+ const DiagnosisPage: React.FC<DiagnosisPageProps>
92+
93+ // ResultPage.tsx - 결과 페이지
94+ interface ResultPageProps {}
95+ const ResultPage: React.FC<ResultPageProps>
96+
97+
98+ ##### 3. UI 컴포넌트
99+ ```typescript
100+ // QuestionCard.tsx - 질문 카드
101+ interface QuestionCardProps {
102+   question: Question;
103+   onAnswer: (answer: number) => void;
104+ }
105+
106+ // ProgressBar.tsx - 진행률 표시
107+ interface ProgressBarProps {
108+   current: number;
109+   total: number;
110+ }
111+
112+ // ResultCard.tsx - 결과 표시
113+ interface ResultCardProps {
114+   result: SkinTypeResult;
115+ }
116+
117+
118+ ##### 4. 서비스 및 훅
119+ ```typescript
```

New Session x

Created requirements.md

요구사항 문서를 작성했습니다. PRD.md의 내용을 바탕으로 8개의 주요 요구사항을 EARS 형식으로 정리했습니다. 각 요구사항은 사용자 스토리와 구체적인 승인 기준을 포함하고 있습니다.

요구사항이 괜찮아 보이나요? 그렇다면 설계로 넘어갈 수 있습니다.

좋습니다! 이제 설계 문서를 작성하겠습니다. 요구사항을 바탕으로 시스템 아키텍처와 구체적인 구현 방안을 설계하겠습니다.

Created design.md

설계 문서를 작성했습니다. FastAPI 백엔드와 HTML/CSS/JavaScript 프론트엔드로 구성된 시스템 아키텍처를 설계했습니다. 주요 특징은 다음과 같습니다:

- 메모리 기반 세션 관리
- RESTful API 설계
- 반응형 웹 인터페이스
- 모듈화된 컴포넌트 구조
- 포괄적인 에러 처리
- 보안 고려사항 포함

설계가 괜찮아 보이나요? 그렇다면 구현 계획으로 넘어갈 수 있습니다.

프론트엔드는 React를 사용하도록 해주세요.

네, React를 사용하도록 설계를 수정하겠습니다.

> 7 edits to file design.md

React를 사용하도록 설계를 수정했습니다. 주요 변경사항:

- 프론트엔드를 React + TypeScript로 변경
- Vite를

> Working..

Cancel

Follow

Ask a question or describe a task...

#

Claude Sonnet 3.7 v Autopilot



MCP Servers: 잘 만들어진 MCP 서버를 연동할 수 있고, 직접 제작한 MCP도 연동 가능합니다. AWS 리소스를 참조해야하는 경우 AWS MCP Registry 활용을 권장합니다.

AI 채팅: 자연어로 Kiro 에이전트에 명령을 전달합니다.

자바개발을 위한 Kiro IDE 셋팅

1 Extension 설정 (OpenVSX 기반)

Extension Pack for Java

VSCode에서 Java 개발에 널리 사용되는 확장 기능들

Spring Boot Extension Pack

Spring Boot 개발을 위한 확장 기능 모음

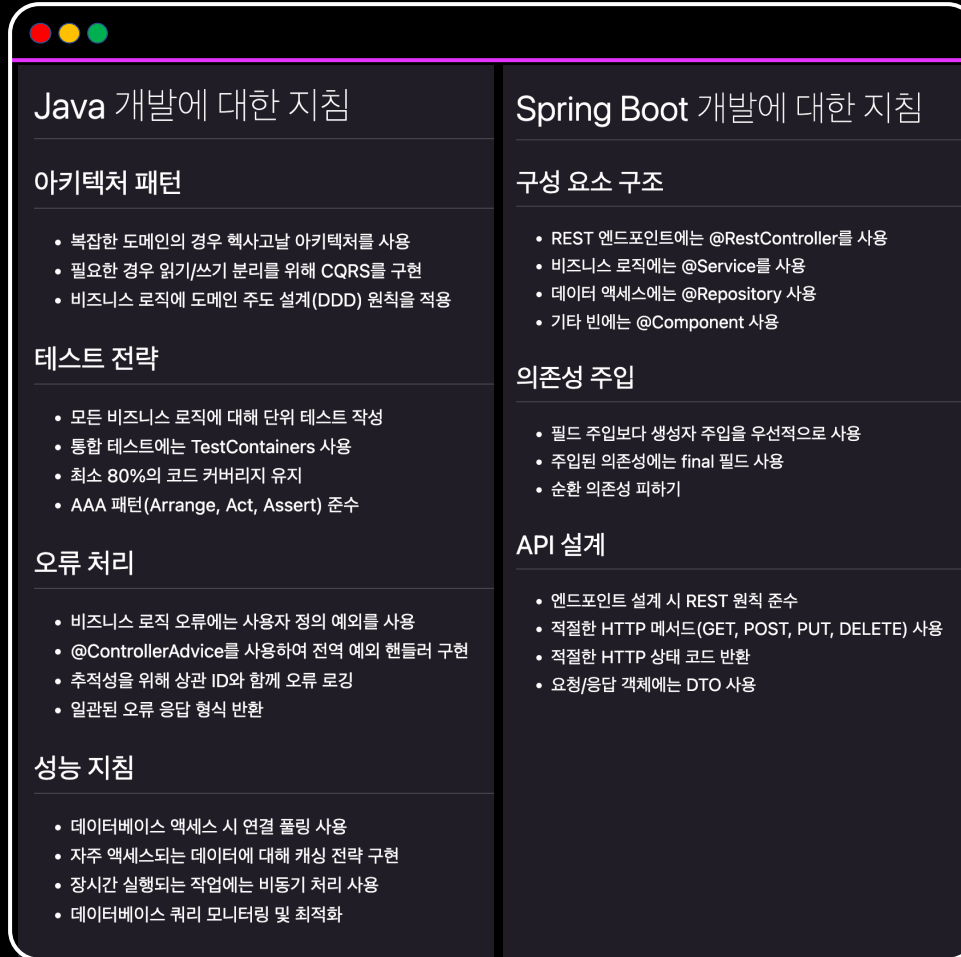
Gradle for Java

Kiro에서 Gradle 프로젝트를 관리 및 실행

Maven for Java

Kiro에서 Maven 프로젝트를 관리 및 실행

2 Kiro Steering(지침) 설정



Java 개발에 대한 지침	Spring Boot 개발에 대한 지침
아키텍처 패턴 - 복잡한 도메인의 경우 헥사고날 아키텍처를 사용 - 필요한 경우 읽기/쓰기 분리를 위해 CQRS를 구현 - 비즈니스 로직에 도메인 주도 설계(DDD) 원칙을 적용	구성 요소 구조 - REST 엔드포인트에는 @RestController를 사용 - 비즈니스 로직에는 @Service를 사용 - 데이터 액세스에는 @Repository 사용 - 기타 빈에는 @Component 사용
테스트 전략 - 모든 비즈니스 로직에 대해 단위 테스트 작성 - 통합 테스트에는 TestContainers 사용 - 최소 80%의 코드 커버리지 유지 - AAA 패턴(Arrange, Act, Assert) 준수	의존성 주입 - 필드 주입보다 생성자 주입을 우선적으로 사용 - 주입된 의존성에는 final 필드 사용 - 순환 의존성 피하기
오류 처리 - 비즈니스 로직 오류에는 사용자 정의 예외를 사용 @ControllerAdvice를 사용하여 전역 예외 핸들러 구현 - 추적성을 위해 상관 ID와 함께 오류 로깅 - 일관된 오류 응답 형식 반환	API 설계 - 엔드포인트 설계 시 REST 원칙 준수 - 적절한 HTTP 메서드(GET, POST, PUT, DELETE) 사용 - 적절한 HTTP 상태 코드 반환 - 요청/응답 객체에는 DTO 사용
성능 지침 - 데이터베이스 액세스 시 연결 풀링 사용 - 자주 액세스되는 데이터에 대해 캐싱 전략 구현 - 장시간 실행되는 작업에는 비동기 처리 사용 - 데이터베이스 쿼리 모니터링 및 최적화	

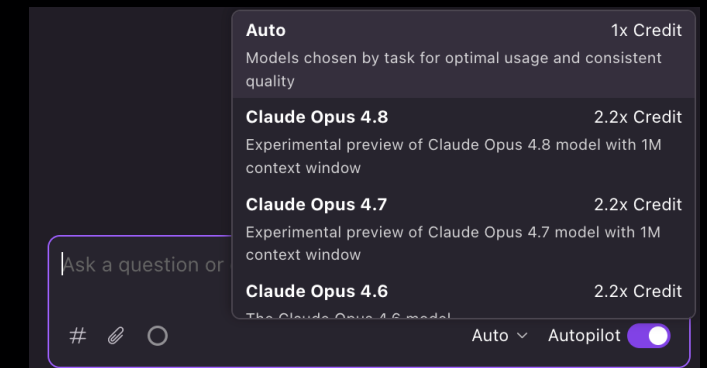
3 Kiro Hook 설정

- Java 파일을 저장할 때 JUnit 테스트를 자동으로 생성
- Checkstyle 또는 SpotBugs를 사용하여 코드 품질 검사
- Outdate 된 Maven 또는 Gradle 종속성 확인
- 공개 메서드에 대한 JavaDoc 주석 생성 또는 업데이트
- Spring Boot 설정 파일 유효성 검사
- Google Java Format 또는 유사 도구로 코드 포맷 지정

4 Kiro MCP 설정

```
{
  "mcpServers": {
    "maven": {
      "command": "uvx",
      "args": ["maven-mcp-server@latest"]
    }
  }
}
```

5 Kiro 모델 설정



Model	Credit
Auto	1x Credit
Claude Opus 4.8	2.2x Credit
Claude Opus 4.7	2.2x Credit
Claude Opus 4.6	2.2x Credit

회사에서 AI 코딩 도구로 자바 개발하기



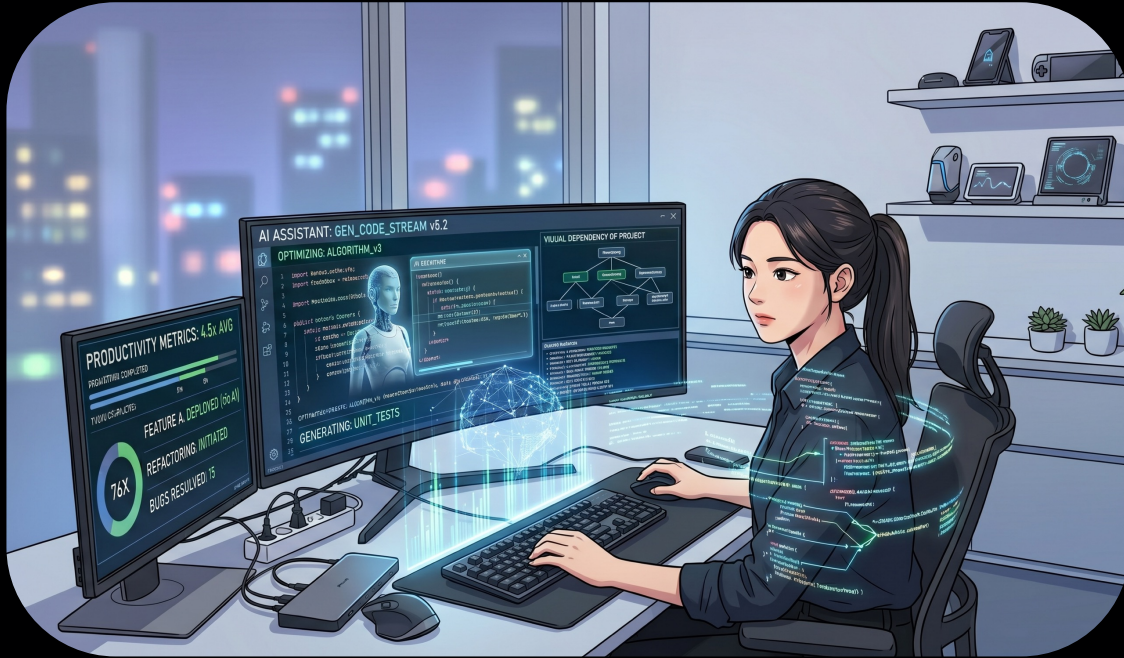
//

소프트웨어 개발에 AI를 활용해도
10-15%의 속도 향상만 달성할 수 있었습니다.

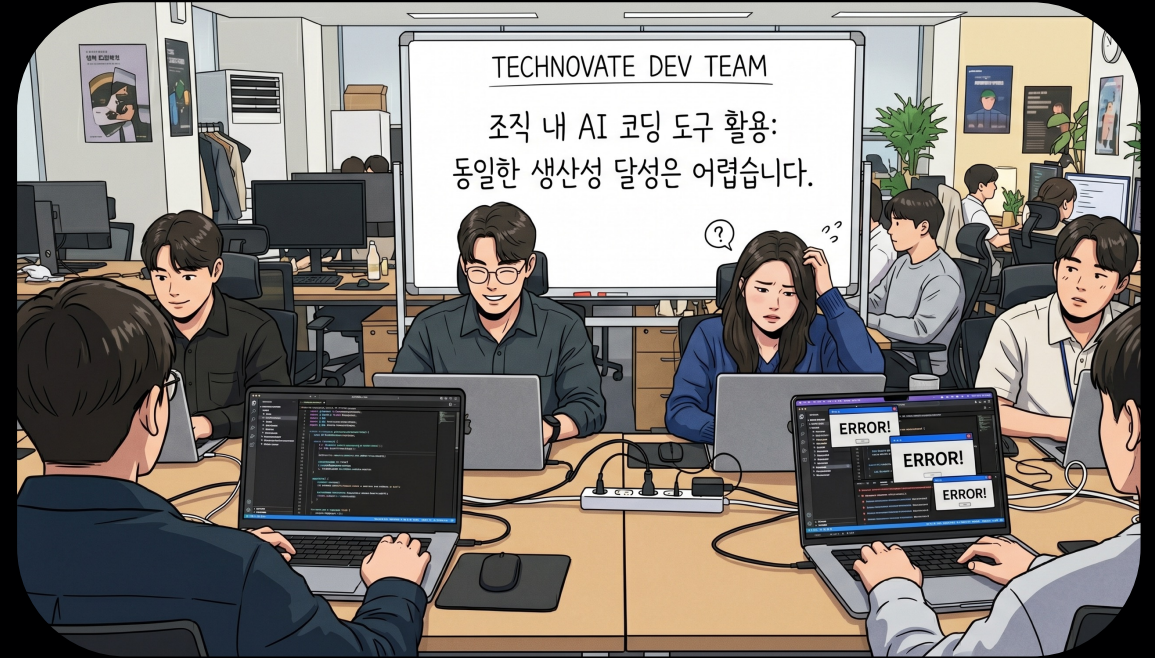


출처: Sichu Zhang, "How much faster can coding assistants really make software delivery?", ThoughtWorks, 2025

개인과 조직 간의 격차



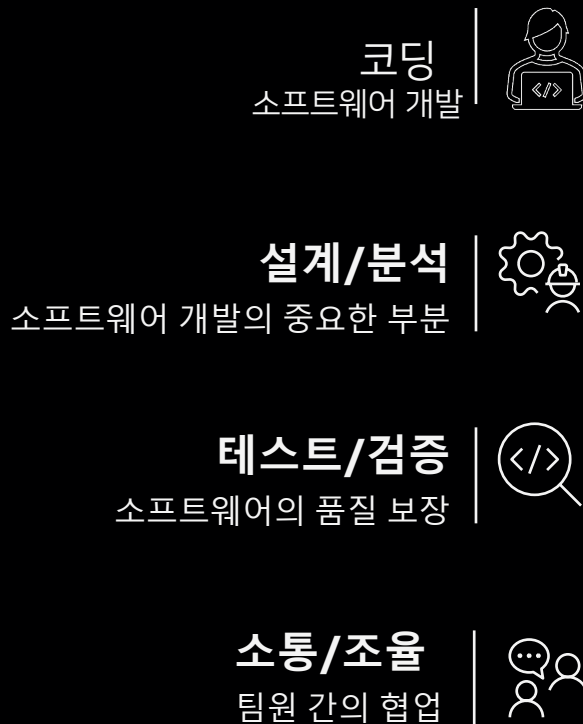
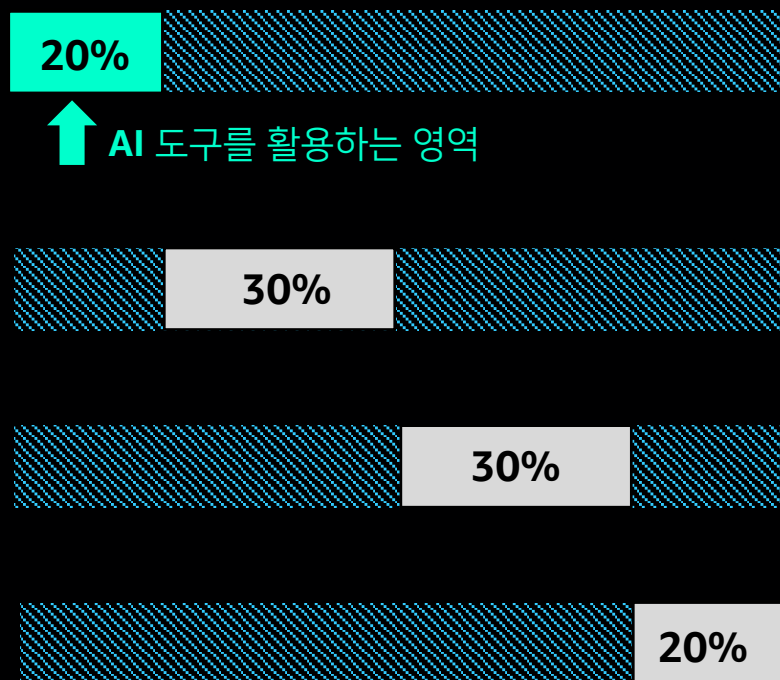
AI를 잘 다루는 **개발자 개인**들은
개인 프로젝트 **생산성 극대화**



회사 내에서는 **AI 활용 능력의 편차**가 존재하여
동일한 생산성을 달성하기 어려움

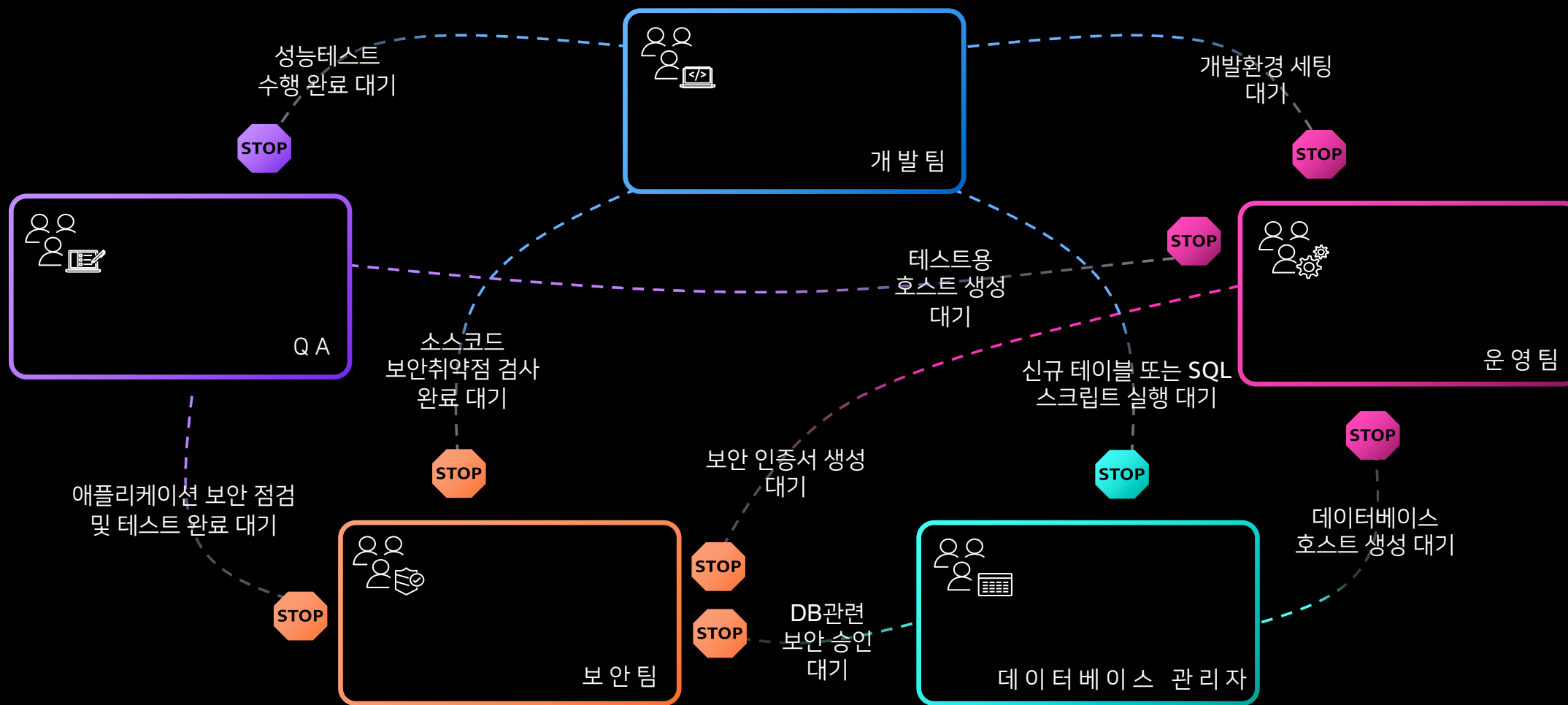
원인 1: AI 도구 활용 범위

<소프트웨어 개발의 업무 구성>



AI 코딩 도구가 코딩 생산성을 50% 향상 시키더라도 전체 생산성은 10% 만 향상

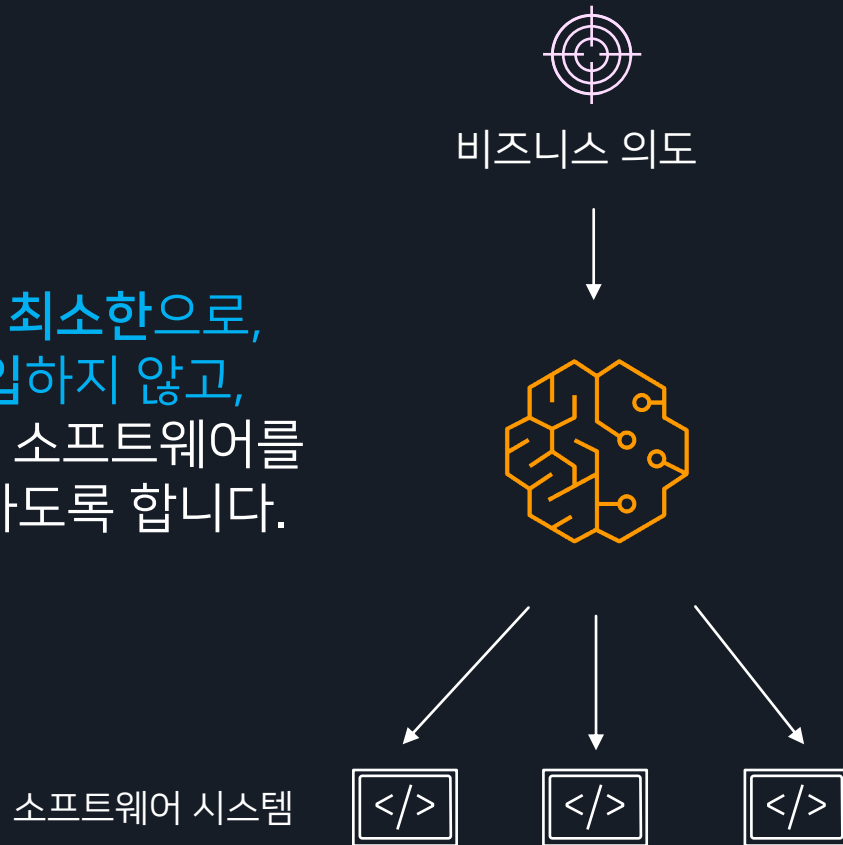
원인 2: AI 도구 활용 편차와 컨텍스트 단절



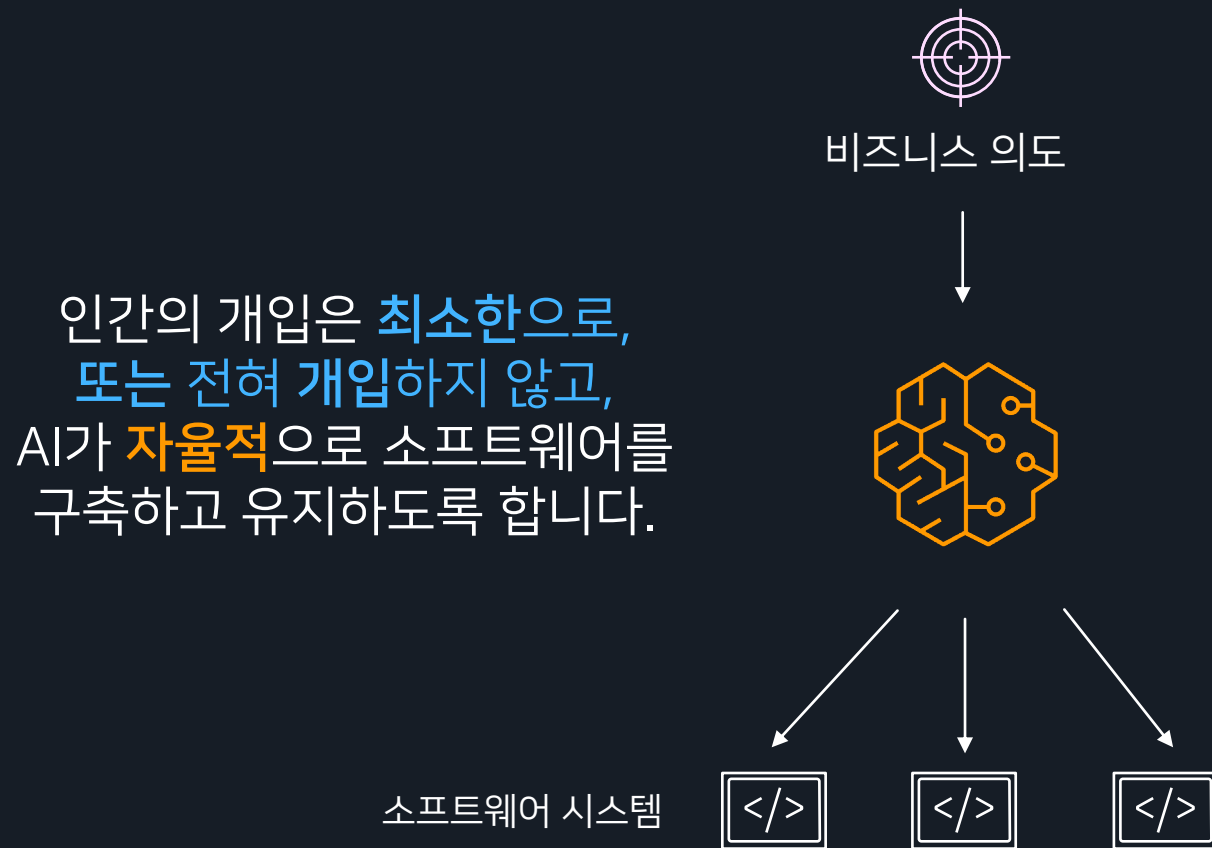
AI 활용 시 흔하게 겪는 개발자들의 2가지 안티 패턴

접근법 1: AI 관리형 (AI-Managed)

인간의 개입은 **최소한으로**,
또는 전혀 개입하지 않고,
AI가 **자율적**으로 소프트웨어를
구축하고 유지하도록 합니다.



접근법 1: AI 관리형 (AI-Managed)



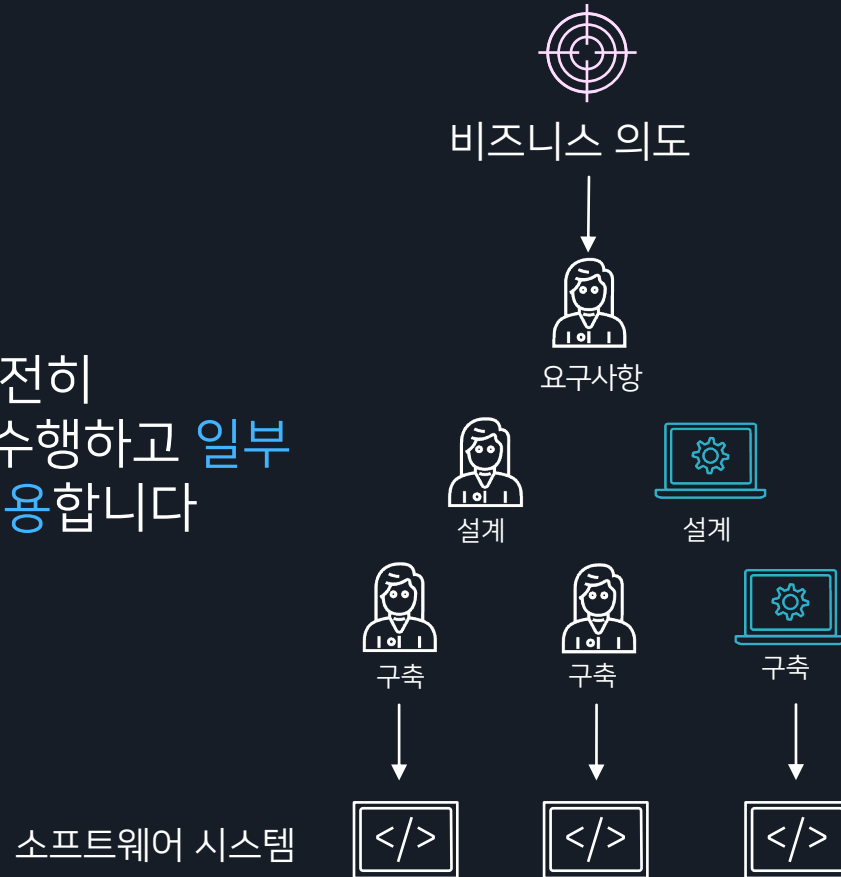
거의 작동하지 않음
신뢰할 수 없고,
설명할 수 없음

개발자가 코드와
"연결"되어 있지 않음

This block is enclosed in a dashed white border. It contains a red warning triangle icon at the top. Below the icon, the text is written in red: '거의 작동하지 않음' (Almost doesn't work), '신뢰할 수 없고,' (Cannot be trusted, and), and '설명할 수 없음' (Cannot be explained). At the bottom, the text is written in white: '개발자가 코드와' (Developer is not connected to the code) and '"연결"되어 있지 않음' (is not connected).

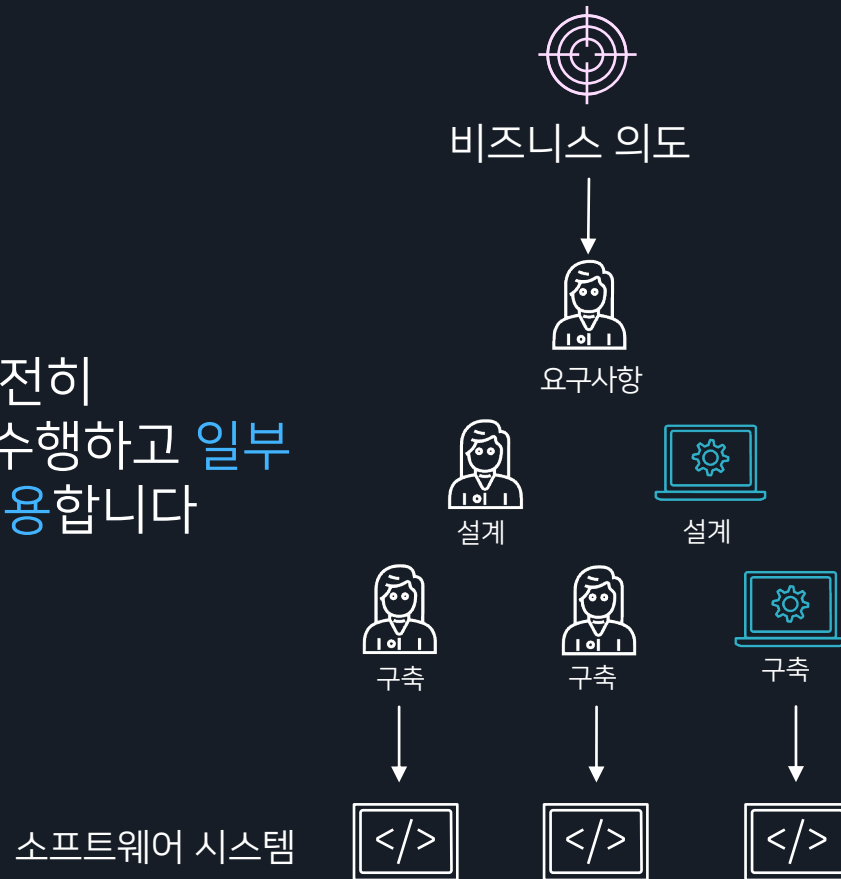
접근법 2: AI 보조형 (AI-Assisted)

개발자들은 여전히
많은 부분 작업을 직접 수행하고 일부
영역에서만 AI를 적용합니다



접근법 2: AI 보조형 (AI-Assisted)

개발자들은 여전히
많은 부분 작업을 직접 수행하고 일부
영역에서만 AI를 적용합니다



AI로 인한 생산성 향상 효과 미미
수동 비효율
기술 부채

코딩에서 절약된 시간은 여전히
SDLC의 다른 요소에서
낭비됩니다.

최근 각광 받는 AI 개발 트렌드



하네스 엔지니어링



AI-DLC
(AI-Driven Development Life Cycle)

하네스 엔지니어링이란?

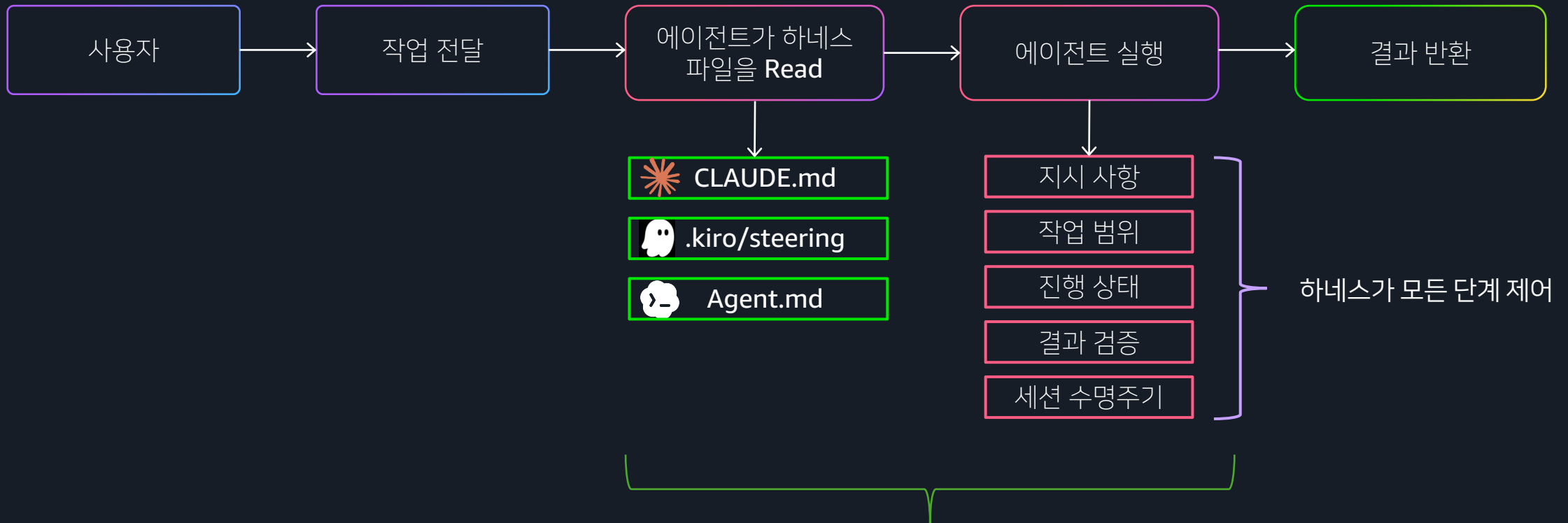


말들을 제어하지 않으면 이리저리 날뛰듯이
생성형 AI 모델도 제어하지 않으면 일관되지
않거나 엉뚱한 답변을 만들어냅니다.



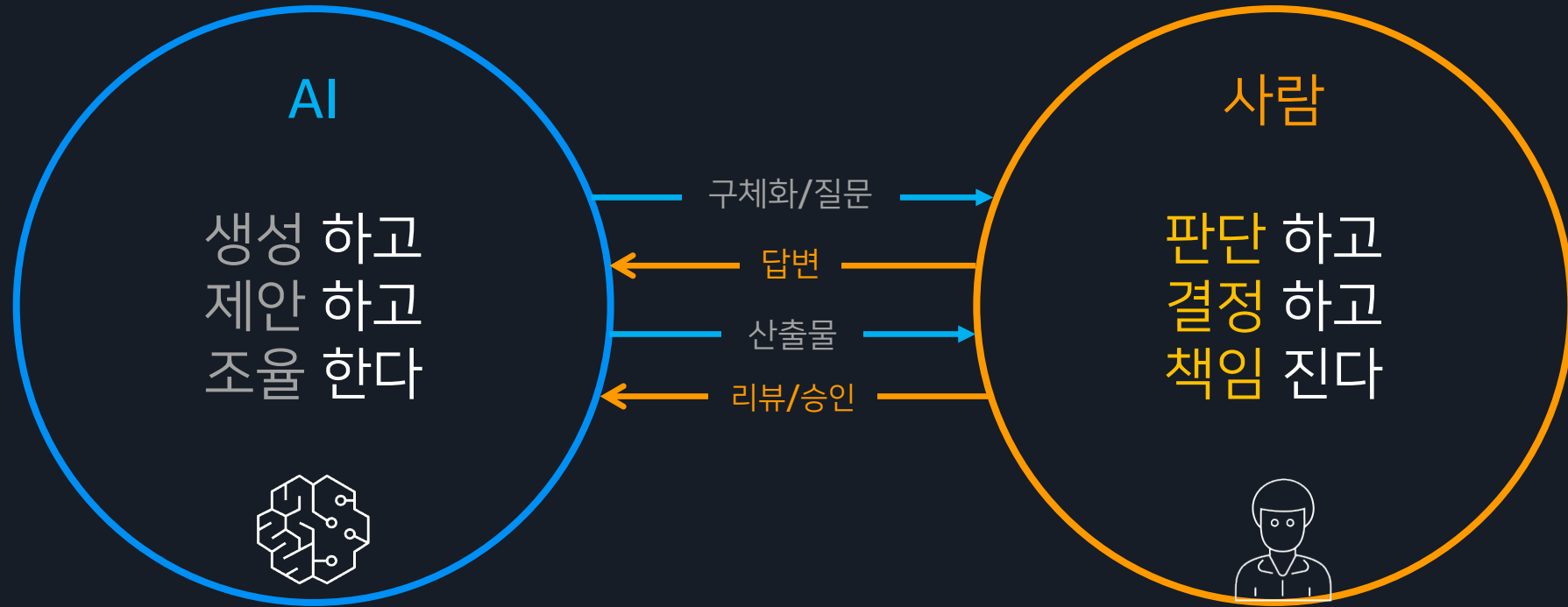
생성형 AI 모델에 하네스를 씌워 우리가
의도한 대로 동작하도록 제어하는 것이
하네스 엔지니어링입니다.

어떻게 동작하는 걸까?



하네스 엔지니어링: 실패를 추적하여 실패한 하네스 단계에 대한 수정 및 재실행 반복
(진단 루프, Diagnostic Loop)

AI-DLC 협업 모델: AI와 사람



“각자 잘하는 것에 집중하고, 워크플로우로 연결한다”

지침을 Kiro steering에 등록하는 것으로부터 시작

Files

main

Go to file

aidlc-rules

- aws-aidlc-rule-details
 - common
 - construction
 - extensions
 - inception
 - application-design.md
 - requirements-analysis.md
 - reverse-engineering.md
 - units-generation.md
 - user-stories.md
 - workflow-planning.md
 - workspace-detection.md
- operations
- aws-aidlc-rules
 - core-workflow.md
 - .markdownlint-cli2.yaml
 - VERSION
- assets
- docs
- scripts
 - .bandit
 - .checkov.yaml
 - .gitignore
 - .gitlints-baseline.json

aidlc-workflows / aidlc-rules / aws-aidlc-rules / core-workflow.md

Preview Code Blame 539 lines (428 loc) · 24.4 KB · 1

Adaptive Software Development Workflow

INCEPTION PHASE

Purpose: Planning, requirements gathering, and architectural decisions

Focus: Determine WHAT to build and WHY

Stages in INCEPTION PHASE:

- Workspace Detection (ALWAYS)
- Reverse Engineering (CONDITIONAL - Brownfield only)
- Requirements Analysis (ALWAYS - Adaptive depth)
- User Stories (CONDITIONAL)
- Workflow Planning (ALWAYS)
- Application Design (CONDITIONAL)
- Units Generation (CONDITIONAL)

Workspace Detection (ALWAYS EXECUTE)

- MANDATORY:** Log initial user request in audit.md with complete raw input
- Load all steps from inception/workspace-detection.md
- Execute workspace detection:
 - Check for existing aidlc-state.md (resume if found)
 - Scan workspace for existing code
 - Determine if brownfield or greenfield
 - Check for existing reverse engineering artifacts
- Determine next phase: Reverse Engineering (if brownfield and no artifacts) OR Requirements Analysis
- MANDATORY:** Log findings in audit.md
- Present completion message to user (see workspace-detection.md for message formats)

타색기

aidlc-spring

core-workflow.md Preview core-workflow.md

INCEPTION PHASE

Purpose: Planning, requirements gathering, and architectural decisions

Focus: Determine WHAT to build and WHY

Stages in INCEPTION PHASE:

- Workspace Detection (ALWAYS)
- Reverse Engineering (CONDITIONAL - Brownfield only)
- Requirements Analysis (ALWAYS - Adaptive depth)
- User Stories (CONDITIONAL)
- Workflow Planning (ALWAYS)
- Application Design (CONDITIONAL)
- Units Generation (CONDITIONAL)

Workspace Detection (ALWAYS EXECUTE)

- MANDATORY:** Log initial user request in audit.md with complete raw input
- Load all steps from inception/workspace-detection.md
- Execute workspace detection:



AI-DLC 지침 로드



Welcome to the new Kiro CLI UX! `/tui` to learn more, `/feedback` for thoughts.
Prefer the classic experience? Relaunch with `kiro-cli --classic`.

● notion requires OAuth – Ctrl+y to copy URL Ctrl+y: Copy URL

Kiro · auto · 8%

ask a question or describe a task ↵

~/Works/workspace/aidlc-spring

/copy to clipboard

INCEPTION 단계 (요구사항 분석 및 설계)

> .aidlc-rule-details
> .kiro
~ aidlc-docs
 ~ inception/requirements
 * requirement-verification-questions.md
 * aidlc-state.md
 * audit.md
> gradle
> history
> src
◆ .gitattributes
◆ .gitignore
📄 build.gradle
≡ gradlew
📄 gradlew.bat
📄 HELP.md
📄 project.zip
📄 settings.gradle

질문 파일 생성

aidlc-docs/inception

각 질문의 [Answer]

질문 요약 :

1. 프론트엔드 기술
2. MSA 서비스 도메인
3. 서비스 간 통신
4. API Gateway / 서비스
5. 데이터베이스 전략
6. 인증/인가 방식
7. 배포 환경
8. 프로젝트 목적
9. 모니터링/관측성
10. 보안 확장 규칙
11. Property-Based

답변 완료 후 알려주

Credits: 1.93 • Time

!kiro .

(no output)

● notion requires OAuth

Kiro · auto · 6%

Ask a question or describe a task...

aidlc-spring

Let's build

Plan, search, or build anything

Vibe

Chat first, then build. Explore ideas and iterate as you discover needs.

Spec

Plan first, then build. Create requirements and design before coding starts.

Great for:

- Rapid exploration and testing
- Building when requirements are unclear
- Implementing a task

Ask a question or describe a task...

@

Auto Autopilot

kspace/aidlc-spring

Autocomplete

Report issue

Kiro Power 1063.39 / 10000 updated just now

/copy to clipboard

AI-DLC 접근 방식

프로젝트 타임라인 →

AI-Managed



사용자는 비즈니스 목표만 설정하고 실행은 AI가 주도하며, 최종 단계에서 인간이 테스트를 완료함

AI-Assisted



사용자가 특정 개별 작업(Task)에 한해 AI를 간헐적으로 활용함

AI-Driven



소프트웨어 개발 전 과정에서 AI가 주도하고 사람이 최종 판단

■ 사람의 개입

■ AI의 개입

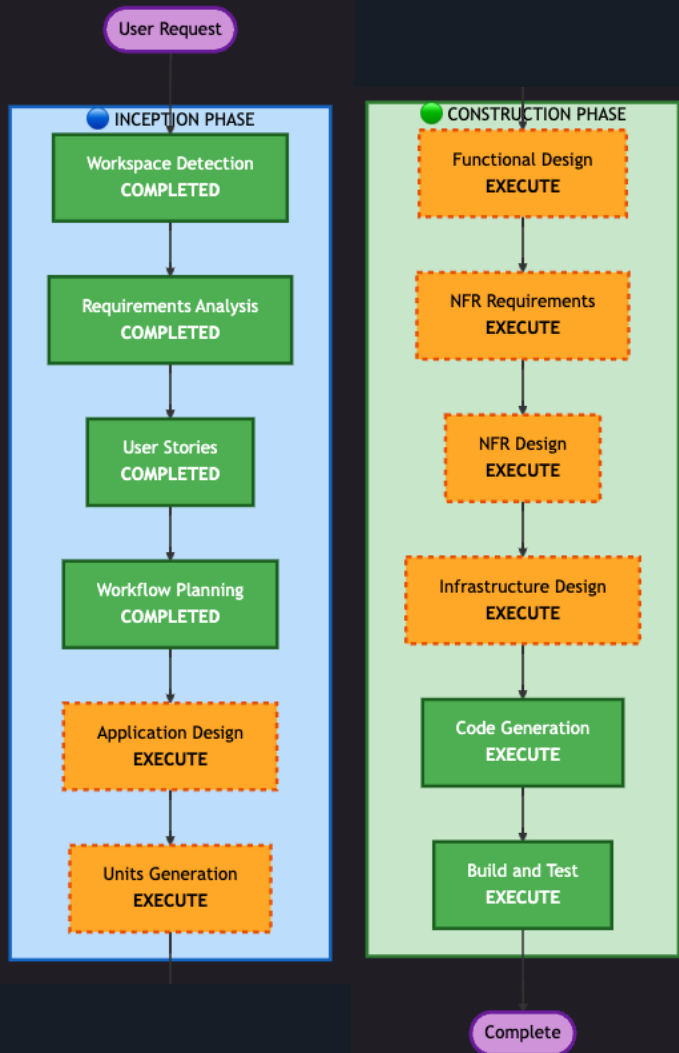
INCEPTION 단계 Flow

실행 계획

요구사항 분석

애플리케이션 설계

사용자 스토리



Requirements Document - Online Shopping Mall

Intent Analysis

- **User Request:** Java Spring Boot 기반 온라인 쇼핑몰, Frontend/Backend 분리, MSA 구조
- **Request Type:** New Project
- **Scope:** System-wide (다수 마이크로서비스 + 프론트엔드 + 인프라)
- **Complexity:** Complex
- **Depth:** Comprehensive

Functional Requirements

FR-001: 회원 서비스 (User Service)

- 회원 가입 (이메일, 비밀번호, 이름, 연락처)
- 로그인/로그아웃 (JWT 기반)
- 회원 정보 조회/수정
- 비밀번호 변경
- 회원 탈퇴
- 배송지 관리 (등록/수정/삭제/목록)

FR-002: 상품 서비스 (Product Service)

- 상품 등록/수정/삭제 (관리자)
- 상품 목록 조회 (페이지, 정렬)
- 상품 상세 조회
- 상품 검색 (이름, 카테고리)
- 카테고리 관리 (계층형)
- 상품 이미지 관리
- 재고 수량 관리

FR-003: 주문 서비스 (Order Service)

- 주문 생성 (상품 선택 → 배송지 입력 → 결제)
- 주문 목록 조회 (사용자별)
- 주문 상세 조회
- 주문 취소
- 주문 상태 관리 (주문접수 → 결제완료 → 배송준비 → 배송중 → 배송완료)
- 주문 시 재고 차감 (Product Service 연동)

FR-004: 결제 서비스 (Payment Service)

System Architecture Diagram



User Stories

분류 방식: 도메인 기반 (서비스별)

우선순위: P1 (필수) > P2 (중요) > P3 (선택)

Epic 1: 회원 서비스 (User Service)

US-101: 회원가입

As a 회원, I want to 이메일과 비밀번호로 회원가입할 수 있다, so that 쇼핑물의 모든 기능을 이용할 수 있다.

Priority: P1

Acceptance Criteria:

1. 이메일, 비밀번호, 이름, 연락처를 입력하여 가입할 수 있다
2. 이메일 형식이 유효하지 않으면 오류 메시지를 표시한다
3. 이미 등록된 이메일로 가입 시도 시 중복 오류를 반환한다
4. 비밀번호는 최소 8자 이상, 영문+숫자+특수문자 조합이어야 한다
5. 가입 성공 시 로그인 페이지로 이동한다

US-102: 로그인

As a 회원, I want to 이메일과 비밀번호로 로그인할 수 있다, so that 인증된 상태로 서비스를 이용할 수 있다.

Priority: P1

Acceptance Criteria:

1. 올바른 이메일/비밀번호 입력 시 JWT 토큰을 발급받는다
2. 잘못된 자격 증명 시 인증 실패 오류를 반환한다
3. 로그인 성공 후 메인 페이지로 이동한다
4. 토큰 만료 시 재로그인을 요구한다

US-103: 회원정보 조회/수정

As a 회원, I want to 내 정보를 조회하고 수정할 수 있다, so that 최신 정보를 유지할 수 있다.

Priority: P2

Acceptance Criteria:

1. 로그인 상태에서 이름, 연락처를 수정할 수 있다
2. 이메일은 변경할 수 없다 (읽기 전용)

CONSTRUCTION 단계 Flow

실행 계획

기능/비기능 요구사항

빌드 및 테스트

Code Generation Plan - User Service

Unit Context

- Unit:** User Service (Unit 2)
- 스토리:** US-101~106, US-501
- 내부 구조:** Layered Architecture (Controller → Service → Repository)
- 인증:** JWT (Access + Refresh Token)
- DB:** PostgreSQL (user_db), Flyway 마이그레이션

Generation Steps

- Step 1: 프로젝트 구조 생성 (build.gradle, settings.gradle, application.yml)
- Step 2: 공통 클래스 생성 (ApiResponse, GlobalExceptionHandler, 커스텀 예외)
- Step 3: 도메인 엔티티 생성 (User, Address, Role)
- Step 4: Repository 생성 (UserRepository, AddressRepository)
- Step 5: JWT 관련 클래스 생성 (JwtTokenProvider, JwtAuthFilter)
- Step 6: Security 설정 생성 (SecurityConfig)
- Step 7: DTO 생성 (Request/Response DTOs)
- Step 8: Service 생성 (AuthService, UserService, AddressService, AdminService)
- Step 9: Controller 생성 (AuthController, UserController, AddressController, AdminController)
- Step 10: Flyway 마이그레이션 스크립트 생성
- Step 11: Dockerfile 생성
- Step 12: 단위 테스트 생성 (Service 레이어)

Business Rules - Product Service

BR-001: 상품 등록 (관리자)

- 상품명, 가격, 카테고리는 필수
- 가격은 0 이상
- 재고 수량은 0 이상 (기본값 0)
- 이미지 URL은 선택

BR-002: 상품 수정 (관리자)

- 모든 필드 수정 가능
- 가격/재고 제약조건 동일

BR-003: 상품 삭제 (관리자)

- Hard Delete
- 진행 중인 주문이 있는 상품은 삭제 불가 (Order Service 확인은 향후 3월)

BR-004: 상품 조회/검색

- 비회원/회원 모두 조회 가능 (인증 불필요)
- 목록 조회: 페이징 + 정렬 (가성순, 최신순)
- 카테고리 필터링
- 상품명 키워드 검색

BR-005: 카테고리 관리

- 2단계 계층 (대분류 → 소분류)
- parentId = null이면 대분류
- parentId != null이면 소분류
- 소분류는 대분류 아래에만 생성 가능 (3단계 이상 불가)
- 하위 카테고리나 상품이 있는 카테고리는 삭제 불가

BR-006: 재고 관리

- 재고 확인: 요청 수량 <= 현재 수량이면 true
- 재고 차감: 현재 수량 - 요청 수량 (음수 불가)
- 재고 복원: 현재 수량 + 복원 수량
- 재고 0이면 상품은 조회 가능하지만 "품절" 표시

NFR Requirements - User Service

Performance

항목	요구사항
로그인 API 응답	평균 200ms 이내
회원 조회 API 응답	평균 100ms 이내
회원 목록 조회 (관리자)	평균 300ms 이내
동시 로그인 요청	100 TPS 이상

Security

항목	요구사항
비밀번호 암호화	BCrypt (strength=10)
Access Token	JWT, HS256, 만료 30분
Refresh Token	JWT, HS256, 만료 7일, Rotation
입력값 검증	Bean Validation (@Valid)
SQL Injection 방지	JPA Parameterized Query
비밀번호 정책	8자 이상, 영문+숫자+특수문자

Availability

항목	요구사항
서비스 등록	Eureka Client 자동 등록
헬스체크	Spring Actuator /health
Graceful Shutdown	지원

Observability

항목	요구사항
로깅	SLF4J + Logback, 구조화 로그 (JSON)

Build and Test Summary

프로젝트 구조 요약

```
aidlc-spring/
├── eureka-server/      # 포트 8761 - 서비스 디스커버리
├── user-service/       # 포트 8081 - 회원/인증
├── product-service/    # 포트 8082 - 상품/카테고리/재고
├── order-service/      # 포트 8083 - 주문 (오케스트레이터)
├── payment-service/    # 포트 8084 - 결제
├── gateway-service/    # 포트 8080 - API Gateway
├── frontend/          # 포트 3000 - React SPA
├── docker-compose.yml  # 전체 오케스트레이션
└── aidlc-docs/        # 문서
```

빌드 순서

- Backend 서비스 Gradle 빌드 (각 서비스 `./gradlew build`)
- Frontend npm 빌드 (`npm install && npm run build`)
- Docker Compose 실행 (`docker compose up --build -d`)

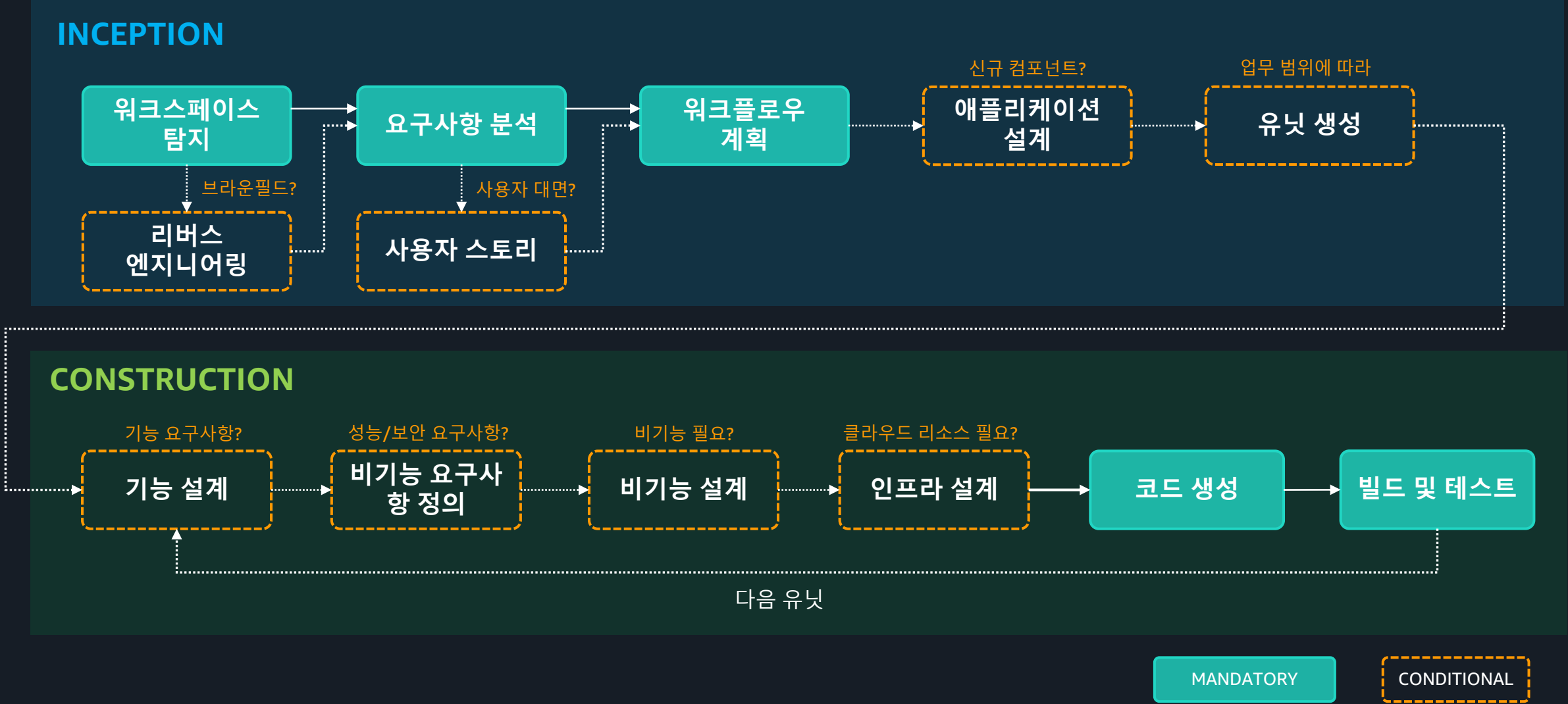
테스트 전략

유형	도구	범위
단위 테스트	JUnit 5 + Mockito	Service 레이어 비즈니스 로직
통합 테스트	Testcontainers	DB 연동 테스트
PBT	jqwik	비즈니스 규칙 속성 검증
E2E 테스트	curl / Postman	API Gateway 통한 전체 플로우

주요 확인 포인트

- 서비스 등록:** Eureka Dashboard에서 5개 서비스 등록 확인
- 인증 플로우:** 가입 → 로그인 → JWT → API 호출
- 주문 오케스트레이션:** 재고 확인 → 차감 → 결제 → 주문 확정
- 보상 트랜잭션:** 결제 실패 시 재고 복원
- 관측성:** Zipkin에서 서비스 간 호출 추적

AI-DLC 워크플로우





최대 50% OFF

SUMMER COLLECTION

시즌 베스트 아이템을 만나보세요



스타일을 완성하는 상의

매일 입고 싶은 편안함과 감각을 담았어요



상의
U.S. Polo Assn. Men Solid
Navy Blue Sweatshirt

207,900원



상의
Puma Navy Blue Men's T-shirt

220,300원



상의
Arrow Sport Men Blue Check
Shirt

282,200원



상의
Lee Men Stripes White T-shirt

217,200원



상의
Highlander Men Green
Checked Halford Slim Fit...

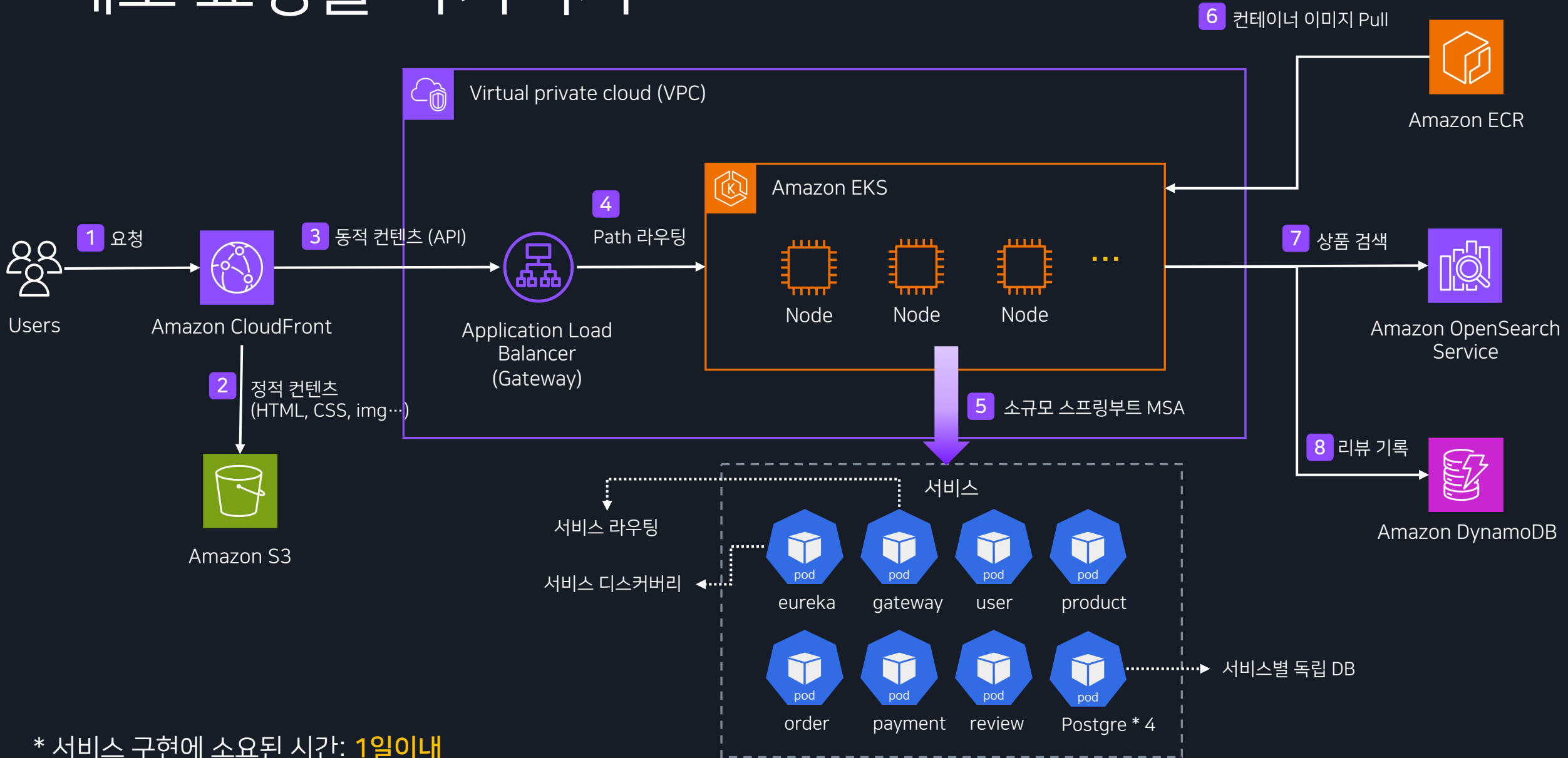
283,500원



상의
Arrow New
Striped Slim Fit...

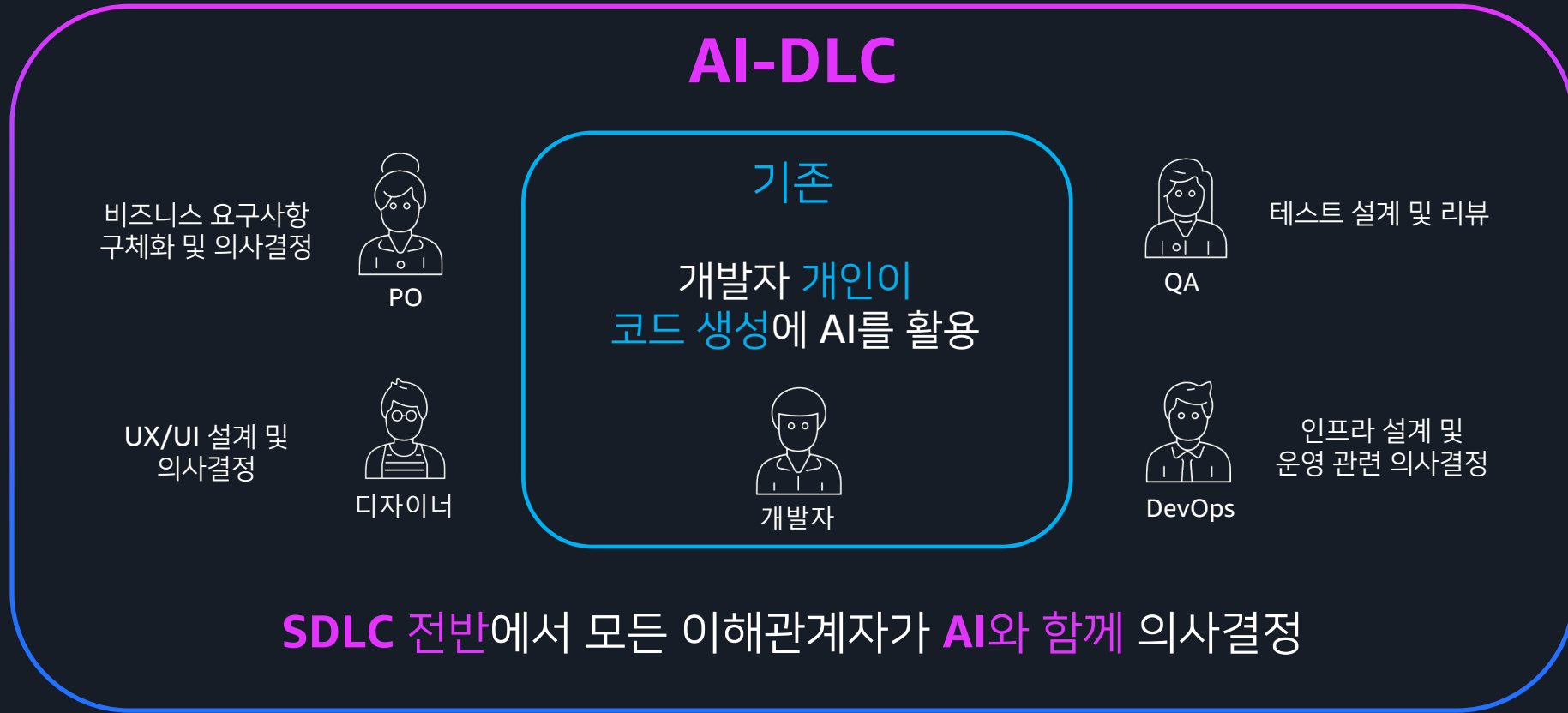
223,500원

데모 쇼핑몰 아키텍처



* 서비스 구현에 소요된 시간: **1일 이내**

AI-DLC 협업 모델: 사람과 사람

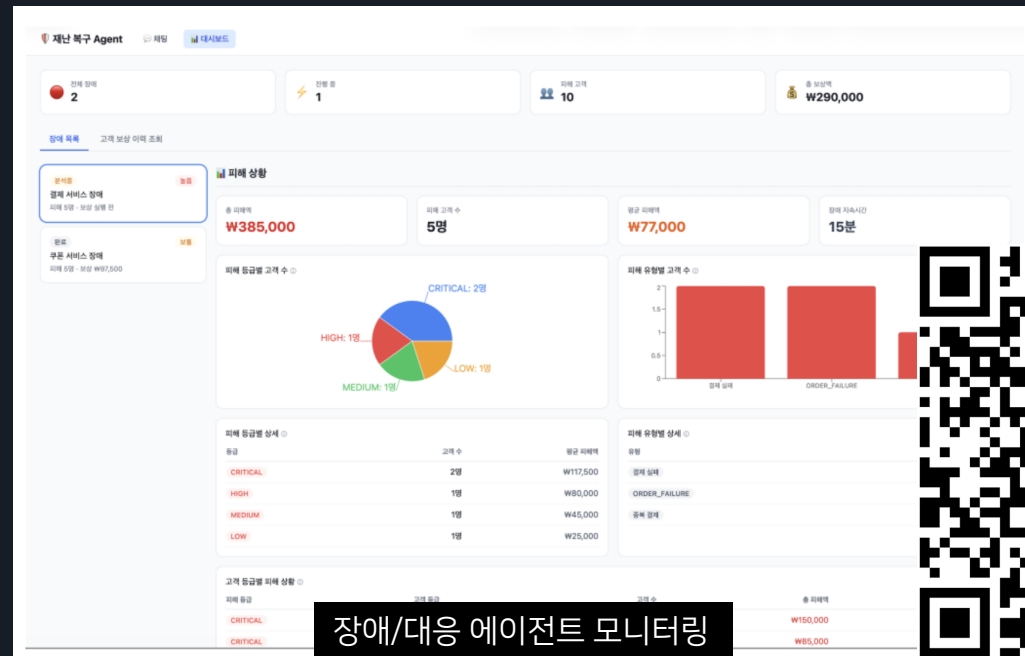
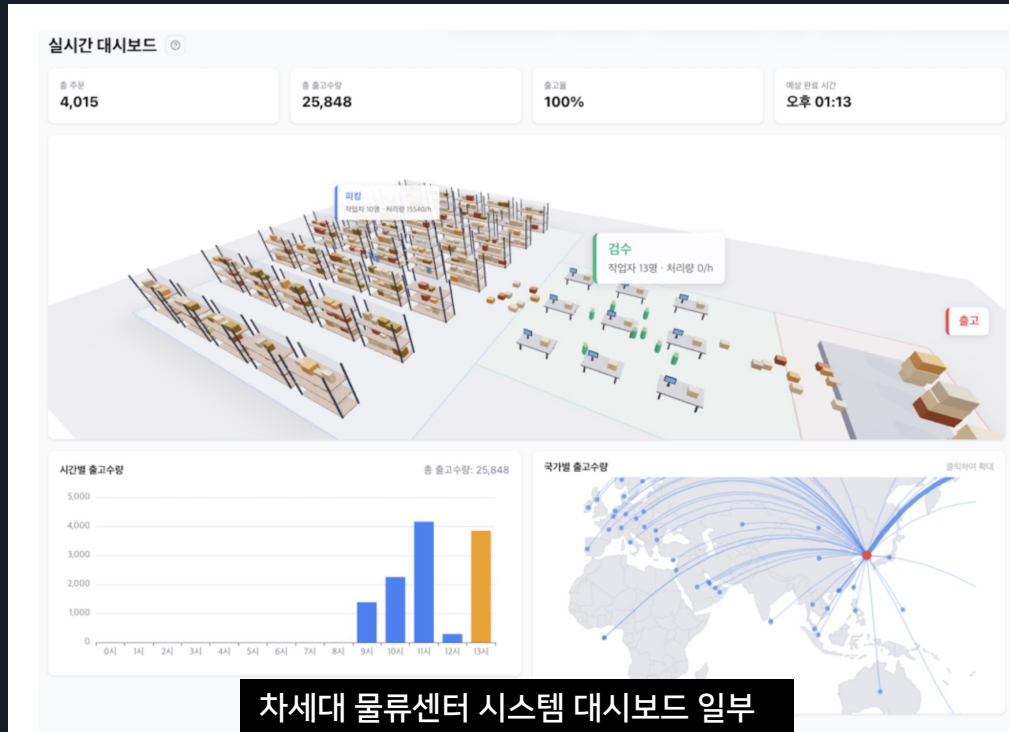
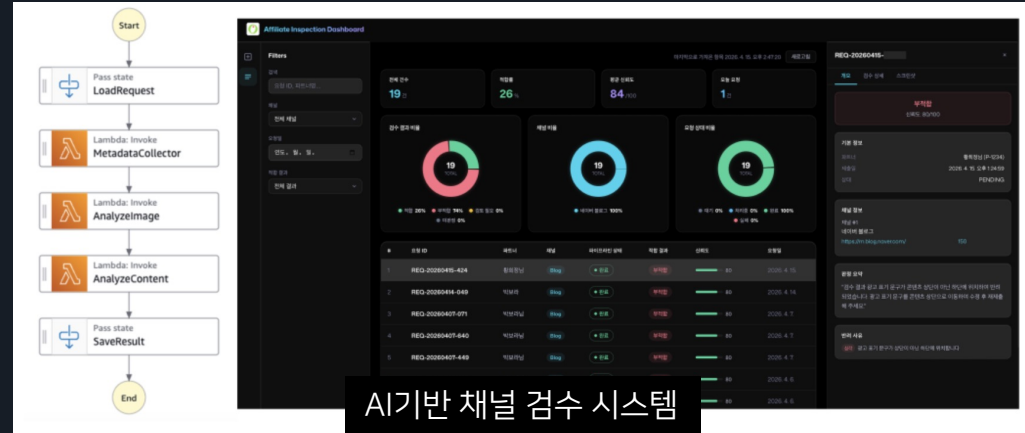
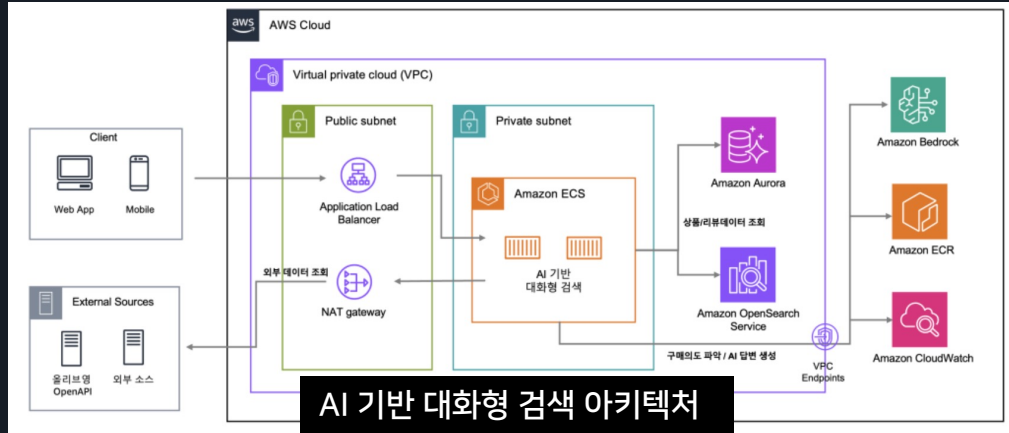


“각자의 전문성으로 함께 의사결정하고, 워크플로우로 연결한다”

AI-DLC 사례: CJ 올리브영



AI-DLC 사례: CJ 올리브영



AWS 자율형 에이전트로 더 효율화 하기



에이전트의 새로운 유형 - 프론티어 에이전트



AWS re:Invent 2025 – Keynote with Matt Garman



Autonomous



Scalable



Long-running

새로운 서비스를 출시해야한다고 생각해봅시다!



소프트웨어 구현

코드를 작성하고, 빌드 및 컨테이너화,
단위 테스트 등

Kiro Web
(Autonomous Agent)



보안

코드 내 잠재적인 위협 식별,
지속적인 점검 등

AWS
Security Agent



운영

배포 자동화, 모니터링, 장애 대응 등

AWS
DevOps Agent

NEW

프리뷰

KIRO

Web

(Autonomous Agent)

1

개발자가 우선순위 높은 업무에 집중하는 동안 다른 업무는 Kiro가 독립적으로 수행

2

세션과 작업 간에 지속적 컨텍스트 유지

3

단일 계획에서 다중 저장소 변경 사항을 일관되게 적용

4

상호작용을 통해 학습하고 적응합니다

Kiro Web

What can I do for you today?

Ask a question or describe a task...

1

YonghoChoi/kiro-msa-frontend X YonghoChoi/kiro-msa-product X

YonghoChoi/kiro-msa-review X YonghoChoi/kiro-msa-search X

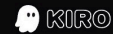
Select repo(s)

Tasks I've been working on

Go to tasks

Tasks (1)

✓ Completed (1) See all >



Autonomous agent (Preview) | Account

email@gmail.com X

Home

Chats

Tasks

Settings

2

We shouldn't have dependencies on the timely and d...

the hydro and hydro-deps repositories.

3:30 PM

I'm working on moving the timely and differential-dataflow dependencies from the hydro repository to the hydro-deps repository while preserving benchmark functionality.

3:48 PM

I'm performing final verifications and then I'll raise this for review.

3:53 PM

I've created the following pull requests for you to review:

- <https://github.com/kyleandmikescazzyadventure/hydro/pull/100>
- <https://github.com/kyleandmikescazzyadventure/hydro-deps/pull/17>

Update the task or ask questions...



Cancel task

We shouldn't have dependencies on the timely and d...

Copy ID

Completed

Repos: kyleandmikescazzyadventure/hydro, kyleandmikescazzyadventure/hydro-

[PR #100](#) [PR #17](#)

3

✓ Creating task - Nov 30, 3:18 PM

Show detailed view

✓ Working on task

✓ Creating pull request(s) [PR #100](#) [PR #17](#)

✓ Completed

Kiro Web – Pull Request

YonghoChoi / kiro-msa-product

Type to search

<> Code

Issues

Pull requests 1

Actions

Projects

Wiki

Security

Insights

Settings

Add comprehensive README.md documentation for product microservice #1

Open

YonghoChoi wants to merge 1 commit into `main` from `kiro-msa-development-20251209-034933`

Conversation 0

Commits 1

Checks 0

Files changed 1

YonghoChoi commented 2 days ago

Owner

Overview

This PR adds comprehensive documentation for the kiro-msa-product microservice.

Requirements Implemented

- ✓ Analyzed the kiro-msa-product repository structure and purpose
- ✓ Created a comprehensive README.md file appropriate for a product microservice
- ✓ Included standard sections such as project overview, setup instructions, usage, and contribution guidelines

Changes Made

- README.md:** Added comprehensive documentation including:
 - Project overview and description
 - Tech stack and architecture details
 - Prerequisites and setup instructions
 - Database schema and models
 - API endpoints documentation
 - Development and deployment guidelines
 - Testing procedures
 - Contribution guidelines
 - License information

Benefits

Reviewers

No reviews

Still in progress? Convert to draft

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request will close these issues.

None yet

Notifications

Unsubscribe

Add comprehensive README.md for product microservice

Co-authored-by: Yongho Choi <yongho1037@gmail.com>

kiro-msa-development-20251209-034933 (#1)

kiro-agent and YonghoChoi committed 2 days ago

commit 3e24395d5e1bb98cc1aa98b5fe2ab19049fd4969

1,159 README.md

<> Viewed

```
@@ -0,0 +1,1159 @@
1 + # Kiro MSA Product Service
2 +
3 + ## Overview
4 +
5 + The Kiro MSA Product Service is a core microservice within the Kiro microservices architecture, responsible for managing the complete product catalog and
6 + lifecycle. This service provides comprehensive product management capabilities including product creation, updates, inventory tracking, categorization, and
7 + detailed product information retrieval.
8 +
9 + As a central component of the e-commerce platform, the Product Service integrates with other microservices:
10 + - **kiro-msa-frontend**: Web interface for product display and management
11 + - **kiro-msa-search**: Product indexing and search functionality
12 + - **kiro-msa-review**: Product reviews and ratings association
13 +
14 + ## Architecture
15 +
16 + ### Microservices Context
17 +
18 + The Product Service follows domain-driven design principles and serves as the source of truth for all product-related data within the Kiro ecosystem.
19 +
20 + ...
21 +
22 +
23 +
24 +
25 +
26 +
27 +
28 +
29 +
30 +
31 +
32 +
33 +
34 +
35 +
36 +
37 +
38 +
39 +
40 +
```

```
graph TD
    Frontend[Kiro MSA Frontend  
(Web Application / Admin)]
    Gateway[API Gateway (optional)]
    Product[Product Service]
    Search[Search Service]
    Review[Review Service]
    Database[(Database  
(Product Catalog))]

    Frontend --- Gateway
    Gateway --- Product
    Gateway --- Search
    Gateway --- Review
    Product --- Database
    Product <--> Search
    Search <--> Review
```

NEW

정식 출시

AWS Security Agent

자신감 있게 배포하세요.
더 안전한 앱을 위한 최첨단 에이전트입니다.

1

애플리케이션을 사전에 보호 조치

2

전문적이고 맞춤화된 보안 가이드 제공

3

원할 때 언제든지 침투 테스트를 수행

AWS Security Agent의 주요 기능들

개발 라이프사이클 전반에 걸친 선제적 보안

설계 검토

몇 분 소요

보안 요구 사항에 따라 요구사항 및 설계 문서를 평가합니다.

코드 검토

준실시간

보안 요구 사항에 따라 풀 리퀘스트(PR) 마다 코드를 평가합니다.

침투 테스트

몇 시간 소요

맞춤형 다단계 공격 시나리오를 통해 검증된 취약점을 발견하고 연결합니다.

애플리케이션이 있는 곳이라면 어디서나 작동합니다



AWS

모든 리전, 모든 엔드포인트
(퍼블릭 또는 프라이빗)



하이브리드

온프레미스
인프라



멀티클라우드

Google Cloud, MS Azure,
Oracle Cloud 등



SaaS

접근 가능한
모든 엔드포인트

모범 사례 기반의 보안 요구사항

보안 요구 사항 정보

보안 요구 사항은 AWS Security Agent가 디자인 리뷰 및 코드 리뷰에서 규정 준수를 분석하는 데 사용됩니다. AWS 관리형 요구 사항 > AWS 보안 에이전트 > 보안 요구 사항

관리형 보안 요구 사항

사용자 지정 보안 요구 사항

활성화된 보안 요구 사항

관리형 보안 요구 사항 (10)

업계 표준에 기반한 AWS 제공 보안 요구 사항으로, 바로 활성화하여 사용자 지정 구성 없이 공통 보안 정책에 대해 평가할 수 있습니다.

이름 또는 설명 기준으로 보안 요구 사항 찾기

☐	보안 요구 사항 이름	상태
☐	Audit Logging Best Practices	활성화됨
☐	Authentication Best Practices	활성화됨
☐	Authorization Best Practices	활성화됨
☐	Information Protection Best Practices	활성화됨
☐	Log Protection Best Practices	활성화됨
☐	Privileged Access Best Practices	활성화됨
☐	Secret Protection Best Practices	활성화됨
☐	Secure by Default Best Practices	활성화됨
☐	Tenant Isolation Best Practices	활성화됨
☐	Trusted Cryptography Best Practices	활성화됨

보안 요구 사항 정보

보안 요구 사항은 AWS Security Agent가 디자인 리뷰 및 코드 리뷰에서 규정 준수를 분석하는 데 사용됩니다. AWS 관리형 요구 사항에서 선택하거나, 각자의 표준에 맞춰 사용자 지정 요구 사항을 생성하세요.

관리형 보안 요구 사항

사용자 지정 보안 요구 사항

활성화된 보안 요구 사항

관리형 보안 요구 사항(10)

업계 표준에 기반한 AWS 제공 보안 요구 사항으로, 바로 활성화하여 사용자 지정 구성 없이 공통 보안 정책에 대해 평가할 수 있습니다.

이름 또는 설명 기준으로 보안 요구 사항 찾기

☐	보안 요구 사항 이름	상태	설명
☒	감사 로깅 모범 사례	활성화됨	시스템이 보안 모니터링을 지원하는지 확인하십시오.
☐	인증 모범 사례	활성화됨	정당한 사용자만 시스템에 접근할 수 있도록 하십시오.

감사 로깅 모범 사례

세부 정보

상태

활성화됨

유형

AWS 관리형

적용 가능성

사용자가 민감한 데이터를 열람하거나 중대한 영향을 미치는 작업을 수행할 수 있는 시스템에 적용됩니다.

설명

시스템이 보안 모니터링을 지원하는지 확인하십시오.

규정 준수 기준

규정을 준수하는 시스템은 보안 모니터링 및 사고 조사에 적합한 세부 수준으로 시스템 변경 사항, 접근 시도 및 사용자 활동을 기록하는 로그를 생성해야 합니다.

규정 준수 여부를 평가하는 데 도움을 주기 위해, 어떤 이벤트가 기록되는지, 각 이벤트에 대해 어떤 정보가 수집되는지, 그리고 로그 기록의 완전성이 어떻게 유지되는지를 설명하는 문서가 마련되어야 합니다. 보안상 중요한 이벤트를 기록하지 못하거나, 로그 항목에서 필수적인 세부 정보를 누락하거나, 조사를 방해할 수 있는 기록 범위의 공백이 있는 경우, 해당 시스템은 명백히 규정을 준수하지 않는 것입니다.

우리 회사 정책에 맞는 보안 요구사항

보안 요구 사항 > 사용자 지정 보안 요구 사항 생성

보안 요구 사항 세부 정보

보안 요구 사항 이름

이 보안 요구 사항에 대한 이름을 추가합니다.

인증 기능 구현

8/80자입니다.

설명

이 보안 확인이 무엇을 적용하는지 간략한 설명을 추가합니다.

프론트엔드와 백엔드에서는 회원가입과 로그인 기능이 필수로 포함되어야 합니다.

43/500자입니다.

적용 가능성

이 보안 요구 사항을 평가해야 하는 시나리오, 시스템 유형 또는 조건을 설명합니다.

고객이 사용하는 서비스에는 반드시 적용되어야 합니다.

29/10,000자입니다.

규정 준수 기준

이 보안 요구 사항에 대해 규정 준수와 비준수를 구성하는 요소를 정의합니다.

로그인 하지 않은 사용자가 서비스를 이용할 수 없도록 해야하며, 서비스 접속 시 로그인이 되어 있는지 확인 후 비로그인 상태이면 로그인 화면으로 전환해야 합니다.

90/10,000자입니다.

수정 지침 - 선택 사항

내부 문서 또는 표준에 대한 링크를 포함하여 위반 사항을 해결하는 방법에 대한 지침을 제공합니다.

수정 지침을 입력합니다.

0/10,000자입니다.

연구 결과 요약

4

규정 미준수

보안 요구 사항의 위반 여부는 지정된 확인 방법의 연구 결과에 따라 결정됩니다.

1

데이터가 부족합니다

4

규정 준수

Findings for: 인증 기능 구현

준수 현황

규정 미준수

보안 요구사항 설명

인증 기능 구현

어떻게

결과가 'NON_COMPLIANT'인 이유는, 시스템이 챗봇 서비스에 API 키 인증을 사용하지만 사용자 가입/로그인 기능이 부족하기 때문이며, 관련 문서에는 인증되지 않은 사용자가 챗봇을 통해 쇼핑 기능에 접근할 수 있다고 명시되어 있기 때문입니다. 3.8절에서는 챗봇이 사용자 인증 없이 쇼핑 관련 질의에 응답하는 모습을 보여주고 있으며, 10절(SEcurity-12)에서는 사용자 로그인이 아닌 API 키 기반 식별 방식을 설명하고 있습니다.

정화 지침

규정 준수를 위해 Express 백엔드에 사용자 등록 및 로그인 엔드포인트를 구현하고, 챗봇 및 쇼핑 기능에 대한 접근을 허용하기 전에 사용자 세션을 확인하는 인증 미들웨어를 추가하며, React 프론트엔드를 수정하여 ChatWidget이나 기타 서비스 컴포넌트를 렌더링하기 전에 인증되지 않은 사용자를 로그인 페이지로 리디렉션할 수 있습니다.

닫기

중앙의 보안팀이 다수의 개발팀과 협업

사내 보안 정책 수립 및 권한 관리
(중앙 보안팀에서 관리)

보안 요구 사항 정보

보안 요구 사항은 AWS Security Agent가 디자인 리뷰 및 코드 리뷰에서 규정 준수를 분석하는 데 사용됩니다. AWS 관리형 요구 사항에서 선택하거나, 각자의 표준에 맞춰 사용자 지정 요구 사항을 생성하세요.

관리형 보안 요구 사항 | 사용자 지정 보안 요구 사항 | 활성화된 보안 요구 사항

관리형 보안 요구 사항 (10)

업계 표준에 기반한 AWS 제공 보안 요구 사항으로, 바로 활성화하여 사용자 지정 구성 없이 공통 보안 정책에 대해 평가할 수 있습니다.

이름 또는 설명 기준으로 보안 요구 사항 찾기

보안 요구 사항 이름	상태	설명
☐ Audit Logging Best Practices	☑ 활성화됨	Ensure the system supports security monitoring.
☐ Authentication Best Practices	☑ 활성화됨	Ensure only legitimate users can access the system.
☐ Authorization Best Practices	☑ 활성화됨	Ensure systems follow authorization best practices.
☐ Information Protection Best Practices	☑ 활성화됨	Ensure sensitive data remains confidential and unaltered.
☐ Log Protection Best Practices	☑ 활성화됨	Protect the integrity and confidentiality of system logs.
☐ Privileged Access Best Practices	☑ 활성화됨	Ensure there are adequate guardrails for privileged functions.
☐ Secret Protection Best Practices	☑ 활성화됨	Ensure that secrets like credentials remain confidential.
☐ Secure by Default Best Practices	☑ 활성화됨	Ensure the system's default configuration is secure.
☐ Tenant Isolation Best Practices	☑ 활성화됨	Ensure appropriate separation between system tenants.
☐ Trusted Cryptography Best Practices	☑ 활성화됨	Ensure only trustworthy cryptographic implementations are used.

보안 점검을 할 수 있는 별도 페이지 제공
(각 개발팀에서 관리)

default에 오신 것을 환영합니다

다음은 AWS Security Agent 스페이스에서 수행한 모든 작업입니다.

▼ 에이전트 스페이스 default 시작하기

설계 검토

설계 문서 및 아키텍처 다이어그램을 업로드하여 조직의 보안 요구 사항에 맞게 평가하세요.

1 설계 검토

침투 테스트

다단계 공격을 사용하여 애플리케이션에 대해 자동화된 침투 테스트를 실행하여 보안 취약성을 식별합니다.

1 침투 테스트

설계 검토

이름으로 설계 검토 찾기

설계 검토 이름 | 컨트롤 | 검토 상태 | 생성된 날짜

ShopEmoji | 10개의 컨트롤 | ☑ 완료됨 | 20 days ago

가장 최근 침투 테스트

shop-emoji-1

대상 URL: https://shop-emoji.amazon.it.kr

마지막 실행: March 19, 2026, 01:34 (UTC+9:00) 완료

최근 3회 실행 결과

완료 March 19, 2026 01:34

0 1 2 3 4 5

중요 높음 중간 낮음

핵심 기능 1: 자동화된 설계 검토 (Design Review)

검토 프로세스



문서 업로드: 아키텍처 설계 문서, API 명세서 또는 회의 녹취록 등을 시스템에 제공합니다.



자동 평가: LLM 에이전트가 정의된 보안 요구사항을 바탕으로 설계의 취약점을 분석합니다.



개선 지침: 발견된 결함에 대해 즉각적인 수정 권고안과 참고 문헌을 제공합니다.



평가 결과 상태

준수 (Compliant)

요구사항을 충족함

부준수 (Non-Compliant)

보안 결함 발견됨

미적용 (N/A)

해당 사항 없음

데이터 부족

추가 정보 필요

"설계 단계에서 보안 문제를 발견하면 사후 수정 대비 비용을 수십 배 절감할 수 있습니다."

데모-1

[클론 설계 검토](#)[삭제](#)[↓ 보고서 다운로드](#)

상세 정보

검토 현황
완료됨

완료일
2026년 3월 18일, 00:52 (UTC+9:00)

검토된 파일
업로드 1건

연구 결과 요약

6

규정 미준수

보안 요구 사항이 위반되었거나 적절히 처리되지 않았습니다.

1

데이터가 부족합니다

업로드된 파일에는 충분한 정보가 포함되어 있지 않습니다.

0

규정 준수

업로드된 파일을 바탕으로 보안 요구 사항이 충족됩니다.

3

해당 없음

보안 요구사항의 적용 기준에 따르면, 해당 요구사항은 이 시스템 설계에는 적용되지 않습니다.

검토된 파일 (1)

파일 이름

▲ | 유형

○ 설계 문서

MD

검토 결과 (10)

아래의 문제점을 선택하여 자세한 내용과 권장되는 해결 방안을 확인하세요.

보안 요구 사항별 검색

규정 준수 상

모두 보기

보안 요구 사항에 따른 검색

정보 보호 모범 사례

권한 부여 모범 사례

검토 결과 (10)

아래의 문제점을 선택하여 자세한 내용과 권장되는 해결 방안을 확인하세요.

보안 요구 사항별 검색

규정 준수 상태별로 필터링

모두 보기

보안 요구 사항에 따른 검색

▼ | 상태

정보 보호 모범 사례

규정 미준수

권한 부여 모범 사례

규정 미준수

기본적으로 보안이 강화된 환경 구축을 위한 모범 사례

규정 미준수

기밀 보호 모범 사례

규정 미준수

감사 로깅 모범 사례

규정 미준수

인증 모범 사례

규정 미준수

로그 보호 모범 사례

데이터가 부족합니다

테넌트 격리 모범 사례

해당 없음

신뢰할 수 있는 암호화 모범 사례

해당 없음

특권 액세스 모범 사례

해당 없음

4

규정 미준수

보안 요구 사항이 위반되었거나 적절히 처리되지 않았습니다.

1

데이터가 부족합니다

업로드된 파일에는 충분한 정보가 포함되어 있지 않습니다.

4

규정 준수

업로드된 파일을 바탕으로 보안 요구 사항이 충족됩니다.

1

검토된 파일

Q Search

파일

○ 선택

검토 결과

아래의 문제점

보안 요구 사항

Q Filter

보안 요구 사항

정보 보호 모드

감사 로깅 모드

기밀 보호 모드

신뢰할 수 있는 암호화 모범 사례

로그 보호 모범 사례

검색 결과: 감사 로깅 모범 사례

×

준수 현황

규정 미준수

보안 요구사항 설명

감사 로깅 모범 사례

어떻게

결과가 'NON_COMPLIANT'인 이유는, 시스템이 민감한 사용자 데이터(인증 토큰, 장바구니, 채팅 세션)와 이에 따른 작업(장바구니 수정, 인증)을 처리함에도 불구하고, 설계 문서에는 보안상 중요한 이벤트에 대한 감사 로깅에 대한 설명이 포함되어 있지 않기 때문입니다. 해당 문서는 오류 응답과 HTTP 상태 코드를 명시하고 있지만, 어떤 보안 이벤트가 로깅되는지, 로그에 어떤 정보가 기록되는지, 또는 보안 모니터링 및 사고 조사를 위해 로깅의 완전성이 어떻게 유지되는지에 대해서는 설명하고 있지 않습니다.

정화 지침

규정 준수를 위해 인증 이벤트(로그인 시도, 토큰 유효성 검사 실패, 토큰 만료), 승인 실패(무단 장바구니/채팅 접근 시도), 민감한 작업(장바구니 수정, 세션 생성)에 대한 감사 로깅을 설명하는 섹션을 추가할 수 있습니다. 각 로그 항목에 어떤 정보(타임스탬프, 사용자 ID, IP 주소, 작업, 결과)가 포함되는지, 그리고 조사를 위해 로그가 어떻게 저장되는지를 명시하십시오.

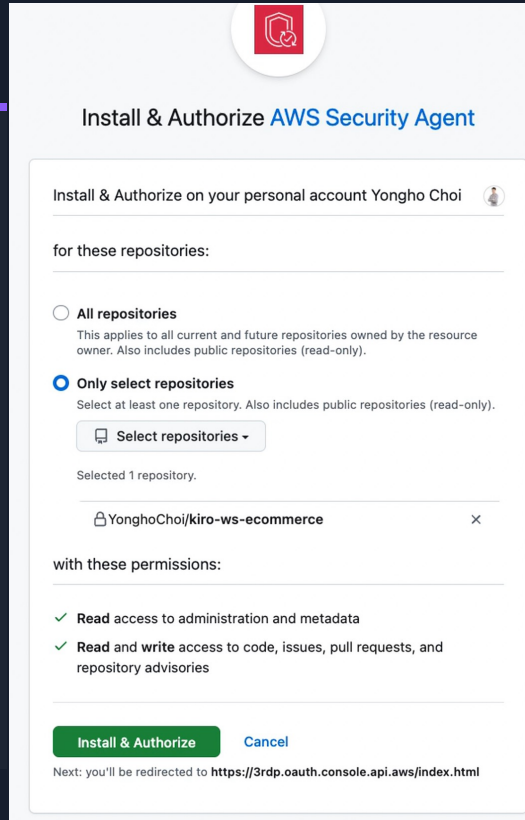
닫기

핵심 기능 2: 실시간 코드 검토 (Code Review)

GitHub 연동을 통해 Pull Request마다 일관된 보안 기준을 자동으로 적용합니다.

자동화된 워크플로우

- GitHub 리포지토리와 네이티브 연동
- 모든 Pull Request 생성 시 자동 보안 스캔 실행
- 코드 변경 사항에 대한 즉각적인 보안 평가



일관된 보안 기준

- 설계 단계의 보안 요구사항을 구현 단계에 그대로 적용
- 조직 전체에 걸친 보안 정책의 일관성 유지
- 수동 검토의 누락 가능성을 제거하고 검토 품질 표준화

PR 생성 후 보안 점검

열기

초기 커밋 #1

YonghoChoi 는 2개의 커밋을 [main에서](#) [develop](#)

↑ rongnochoi 2건의 커밋 추가 2026년 3월 18일 23:00

feat: add shopping assistant chatbot with auth, cart isolation, and c...

059784b

chore: update .gitignore to exclude __pycache__, db-shm/wal files

d368a97



aws-security-agent (bot) 댓글 2026년 3월 18일

AWS Security Agent가 귀하의 풀 리퀘스트를 검토 중이며, 곧 피드백을 남겨드릴 예정입니다.



aws-security-agent (bot) 검토됨 2026년 3월 18일

검토된 변경 사항 보기

aws-security-agent (bot) 댓글 남기기

이 PR에는 병합 전에 반드시 해결해야 할 몇 가지 중대한 보안 취약점이 있습니다. 가장 시급한 문제는 클라이언트 측 자바스크립트 번들에 API 키(VITE_CHAT_API_KEY)가 노출되어 있어 챗봇 엔드포인트의 인증이 완전히 무력화되는 점과, SessionManager에 사용자별 세션 키가 없어 공격자가 세션 ID를 추측하여 다른 사용자의 대화 내역에 접근할 수 있다는 점입니다. 또한, 백엔드의 CORS 구성은 인증된 경로를 포함한 모든 엔드포인트에 대해 개방형 와일드카드 정책을 사용하고 있으며, CSRF 보호 기능이 전혀 구현되어 있지 않습니다. 이 두 가지 모두 애플리케이션이 발전함에 따라 심각한 위험을 초래할 수 있습니다. 코드베이스 전반에 걸쳐, 장바구니 수량 처리 및 챗봇 도구 출력 부분의 입력 검증 및 세척(sanitization) 결함도 비즈니스 로직 악용 및 잠재적인 XSS 공격을 방지하기 위해 수정되어야 합니다.



tend/src/pages/ProductPage.jsx

```
+ <div className="add-section">
+   <label htmlFor="qty">Qty:</label>
+   <input
+     id="qty"
+     type="number"
+     min={1}
+     max={99}
+     value={qty}
+     onChange={(e) => setQty(Math.max(1, +e.target.value))}>
```

▼ R53 행에 대한 댓글

aws-security-agent[봇] 4 minutes ago

문제는 무엇인가요? 수량 필드는 onChange 핸들러 내의 `Math.max(1, +e.target.value)` 와 HTML 속성인 `min={1}` `max={99}` 를 통해 최소값을 1로 강제하지만, 이는 클라이언트 측에서만 적용되는 제어 방식입니다. qty 상태 변수는 React 세터를 직접 호출하거나 DOM을 조작하여 임의로 큰 값이나 음수 값으로 설정될 수 있으며, 이 컴포넌트에서는 서버 측 유효성 검사 가 보이지 않는 상태에서 원시 qty 값이 27행의 `addToCart(product.id, qty, token)` 으로 직접 전달됩니다. 백엔드 `addToCart` 엔드포인트가 수량 범위를 독립적으로 검증하지 않는다면, 공격자는 음수 수량(장바구니 총액을 감소시키거나 재고가 마이너스가 되는 상황을 유발할 수 있음)이나 극도로 큰 수량을 제출할 수 있습니다.

이것이 왜 중요한가요? 클라이언트 측 입력 제약 조건은 쉽게 우회될 수 있습니다. 수량 제한을 위해 이러한 컨트롤에만 의존하는 비즈니스 로직은, 사용자가 직접 API 요청을 작성하거나 React 상태를 조작함으로써 무력화될 수 있습니다.

권장 사항은 무엇인가요? 프론트엔드 검증과 별개로, 백엔드 장바구니 엔드포인트에서 서버 측에서 수량 제한(예: 1-99)을 적용하십시오.

이 댓글에 👍 또는 🙋를 눌러 의견을 남겨주세요



답글 쓰기

댓글 해결

+

/>

핵심 기능 3: 지능형 침투 테스트 (Penetration Testing)

컨텍스트를 인식한 자율 침투 테스트



AI 기반 침투 테스트

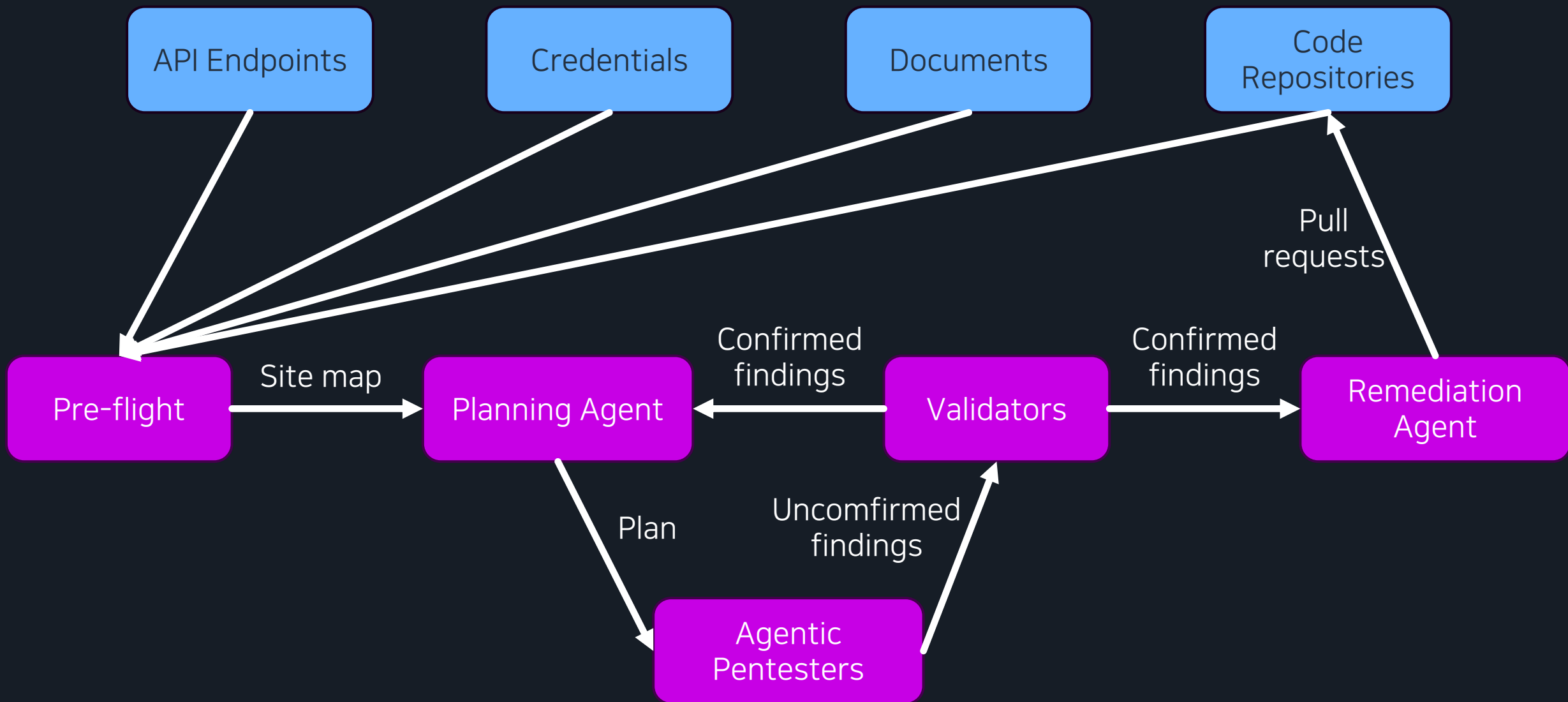
SAST, DAST, SCA 및 자율 침투 테스트를 단일 솔루션으로 통합합니다.

컨텍스트 인식

설계 문서, API, IaC, 위협 모델, 런타임 동작 및 사용자 스토리를 수집하여 애플리케이션에 대한 다단계 공격 계획을 수립합니다.

검증된 보안 위험

취약점을 검증되고 재현 가능한 공격으로 연결하여 오탐지 및 경보 피로를 줄입니다.



침투 테스트 결과 확인 Demo

“침투 테스트 로그”로 SI
가 어떤 작업을 수행했는
지 확인

“조사 결과”를 통해 취약
점 분석

조치 후 “재현 단계”를 참
고하여 재발 여부 검증



AWS Security Agent Administrator

홈 > default > 침투 테스트 > shop-emoji-1 > March 19, 2026 penetration test run

March 19, 2026 침투 테스트 실행 Completed

침투 테스트: shop-emoji-1 마지막 업데이트: March 19, 2026, 04:36 (UTC+9:00) 기간: 3 hours 2 minutes 21 seconds

↓ 보고서 생성 실행 중지

Completed
프리플라이트

Completed
정책 분석

Completed
침투 테스트

Completed
마무리 중

침투 테스트 실행 개요 침투 테스트 로그 조사 결과

작업(48)

AWS Security Agent가 식별하고 수행한 작업은 침투 테스트 실행이 진행됨에 따라 여기에 표시됩니다.

🔍 침투 테스트 작업 필터링

▼ Setup infrastructure (2)

Logging infrastructure setup

성공

작업이 시작됨: March 19, 2026, 01:35 (UTC+9:00)

Setting up logging infrastructure with CloudWatch log groups, log streams and S3 for pentest execution...

Setup testing environment

성공

작업이 시작됨: March 19, 2026, 01:35 (UTC+9:00)

Setting up secure testing environment for pentest execution. Testing environment setup completed successfully...

▶ Preflight validator (2)

▶ Login (2)

▶ Network scanner (3)

▶ Cross site scripting (4)

▶ Local file inclusion (2)

Logging infrastructure setup 로그

시작 시간 March 19, 2026, 01:35 (UTC+9:00)	상태 Completed
기간 < 1s	작업 그룹 Setup infrastructure

설명
Setting up logging infrastructure with CloudWatch log groups, log streams and S3 for pentest execution. Logging infrastructure was created successfully.

작업 로그

로그 이미지 보기

사용 가능한 로그 없음

이 작업은 성공적으로 완료되었습니다. 이 작업 유형에는 로그가 생성되지 않습니다.

© 2026, Amazon Web Services, Inc. 또는 자회사. All rights reserved. 개인 정보 보호 약관 쿠키 기본 설정

NEW

정식 출시

AWS DevOps Agent

운영 우수성을 위한 최첨단 에이전트입니다.

1

상시 인시던트 분류 및 가이드 기반 대응

2

기존 모니터링 도구에서 인사이트 확인
(Dynatrace, New Relic, Datadog, Splunk, CloudWatch)

3

리소스 간 관계를 지능적으로 매핑해
MTTR(Mean Time To Repair)을 줄이고
서비스 안정성을 강화

Customer challenges



애플리케이션 복잡성 증가



텔레메트리가 여러
도구로 확산



분산 시스템에서의
파편화된 전문성



급한 불 부터 끌 것인가
체계적으로 개선할 것인가

+ 새 채팅

📄 페이지

인시던트 대응

개선 사항

아티팩트

토폴로지

🗨 채팅

모두 보기

👤 에이전트

⚡ 스킬

⚙ 설정

👤 Participant
devops-agent-space

오늘은 어떻게 도와드릴까요?

무엇을 할 수 있나요?

최근에 무엇을 했나요?

상태 요약을 알려주세요.

DevOps 에이전트와 채팅하기



DevOps Agent의 핵심 기능

조사 및 완화

- 다양한 도구와 연동하여 데이터를 수집
- 근거를 제시하며 근본 원인을 규명
- 안전한 완화 조치를 제공

MCP 서버로 맞춤 제어

- 보유한 MCP 도구를 자유롭게 연결
- 사용 중인 오픈서빌리티 서비스를 그대로 활용
- Kiro, Bedrock AgentCore Gateway, AWS Lambda, CDK를 활용하여 MCP 서버를 손쉽게 배포

SKILLS, 채팅으로 조사 방향 제어

- 자신의 인사이트로 조사에 영향
- 채팅을 통해 환경에 대한 정보를 파악
- Skills로 향후 조사를 가속화

문제 사전 예방

- 운영 체계의 개선점을 발견
- 반복적으로 발생하는 문제 해결
- 재발하는 이슈를 예방하고, 가장 빈번한 문제부터 우선 해결

AWS DEVOPS AGENT

팀의 일원으로서 함께 일합니다



PagerDuty

servicenow®

- ServiceNow, PagerDuty 등 기존 도구의 티켓과 알람에 자동으로 대응합니다.
- 조사 결과를 트리거된 티켓과 Slack 등 팀 협업 채널에 게시합니다.
- Dynatrace Davis 등 다른 AI 에이전트와 협력하여 작동합니다.



AWS DEVOPS AGENT

텔레메트리 전문가



dynatrace



DATADOG



new relic®

splunk>
a CISCO company

- AWS CloudWatch, Dynatrace, Datadog, New Relic, Splunk과 기본 연동
- MCP 서버를 통해 Grafana, Prometheus 등 모든 텔레메트리 제공자로 확장 가능
- 알람, 메트릭, 로그, 트레이스를 조회하여 근본 원인 분석(RCA)을 수행하고 옴저버빌리티 개선을 권장

AWS DEVOPS AGENT

파이프라인 전문가

GitHub



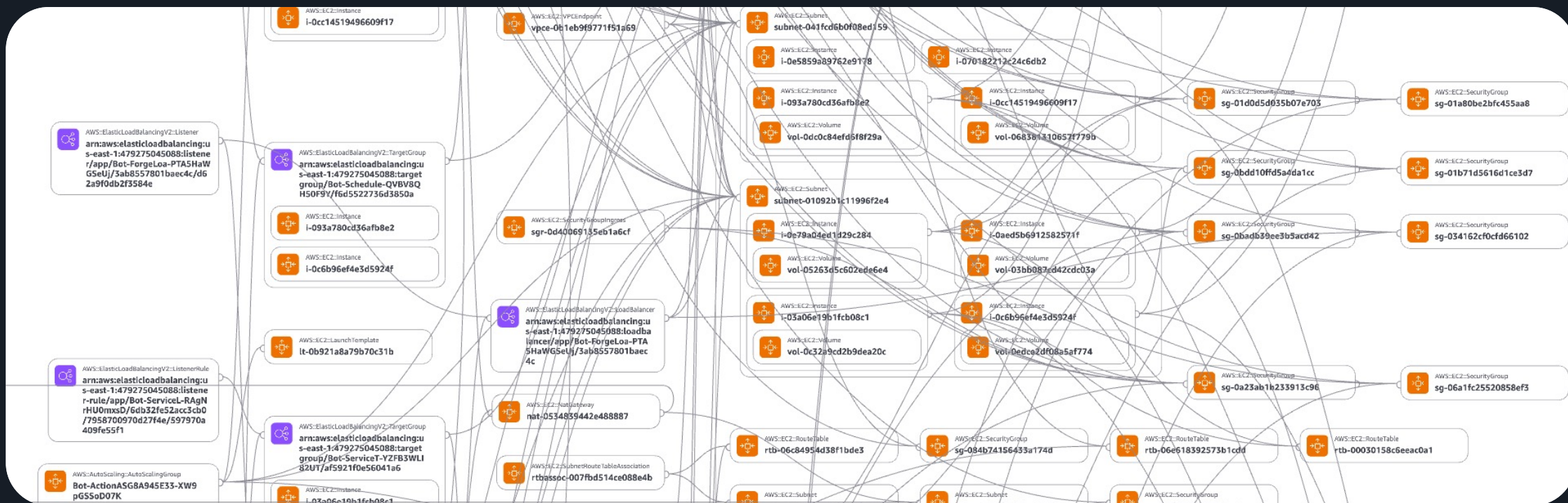
GitLab



Azure DevOps

- GitHub, GitLab, Azure Devops와 기본 연동
- 인시던트를 유발한 코드 배포를 식별하고, 롤백 가이드를 제공
- 향후 인시던트 방지를 위한 파이프라인 개선을 권장

이 복잡한 어플리케이션 토폴로지 뷰를 사람이 파악하기는 어렵지만, AWS DevOps Agent는 할 수 있습니다!



이슈 대응과 운영시간의 획기적인 변화

10개 이상의 고객과 파트너의 사례



MTTR 의 감소

75%

평균

범위: 50-92%



조사 시간

80%

더 빠르게

42 분 → 인시던트 당 8분



원인 분석의 정확도

94%

평균

범위: 84-100% 의 정확도



이슈 해결 속도 개선

8.5x

더 빠르게

범위: 7-10x 더 빠르게



생산성 개선

57 시간/월

평균 소요 시간/월 절감 시간

범위: 22-120 시간/월 절감시간



Detailed customer
and partner quotes



Wrap up



소프트웨어 구현

AI-DLC로 조직 전반에
일관된 코드 품질 달성
Kiro Web으로 자율적인 코딩



보안

Security Agent로
초기부터 보안 점검



운영

DevOps Agent로 장애 발생 시
빠르게 Root Cause 식별

더 나은 콘텐츠를 위해 **피드백**을 남겨주세요



설문 완료 후 **발표자료**를 다운 받으실 수 있습니다 😊



감사합니다.

최용호
테크에반젤리스트
AWS

